

# 基于UDP的TFTP文件传输项目

## 一、TFTP传输模型的介绍

基于UDP通信的TFTP（Trivial File Transfer Protocol，简单文件传输协议）模型是一种用于在网络上进行文件传输的简单协议。TFTP设计简单，易于实现，通常用于传输小文件或在不支持复杂协议的环境中使用

### 1.1 协议概述

TFTP使用UDP（User Datagram Protocol，用户数据报协议）作为传输层协议，默认端口号为69。TFTP协议设计简单，仅支持五种类型的报文，分别是：

- **RRQ (Read Request)**：客户端请求读取文件。
- **WRQ (Write Request)**：客户端请求写入文件。
- **DATA**：数据传输报文。
- **ACK**：确认报文。
- **ERROR**：错误报文。

### 1.2 通信流程

TFTP的通信流程通常分为两个阶段：请求阶段和传输阶段

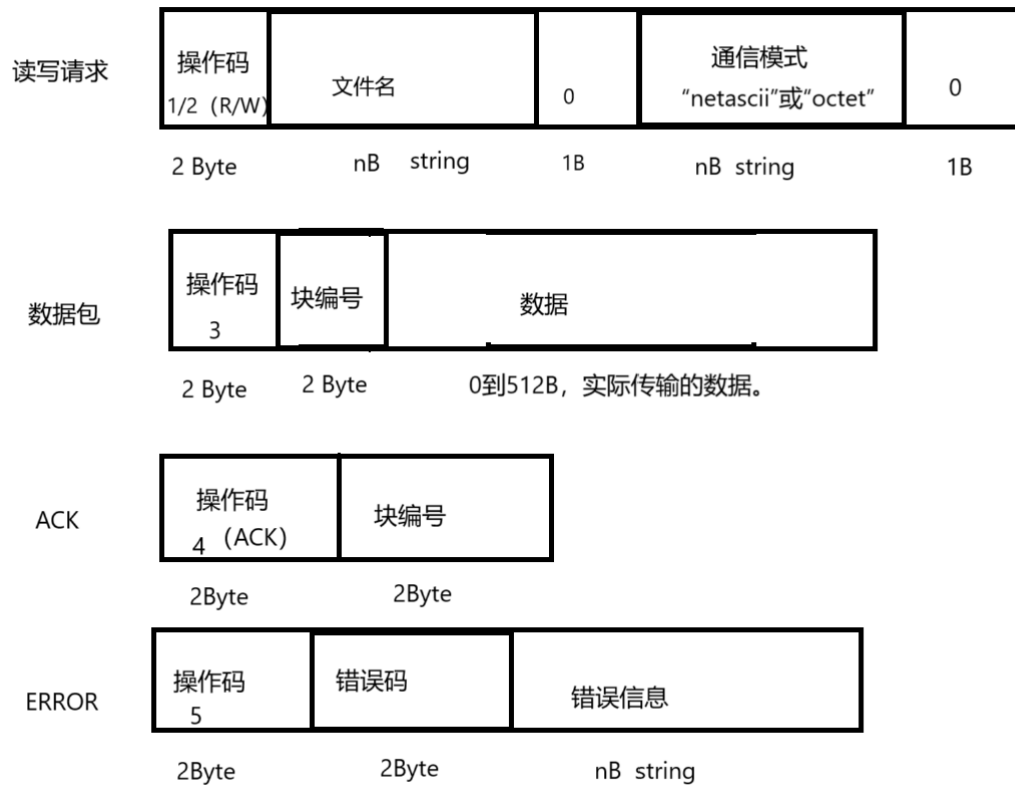
#### 1.2.1 请求阶段

- **客户端发送RRQ或WRQ**：客户端向服务器发送RRQ或WRQ报文，请求读取或写入文件。RRQ报文包含文件名和传输模式（通常是“netascii”或“octet”），WRQ报文类似。
- **服务器响应**：服务器接收到RRQ或WRQ后，会分配一个新的UDP端口用于数据传输，并发送ACK（对于WRQ）或直接发送DATA（对于RRQ）作为响应。

#### 1.2.2 传输阶段

- **数据传输**：在RRQ请求中，服务器开始发送DATA报文，每个DATA报文包含一个块编号和数据块。客户端接收到DATA报文后，发送ACK报文确认接收到的块编号。在WRQ请求中，客户端发送DATA报文，服务器发送ACK报文确认。
- **块编号**：每个DATA报文和ACK报文都包含一个16位的块编号，从1开始递增。块编号用于确保数据的有序传输和确认。
- **结束传输**：当传输的文件大小小于512字节时，DATA报文的数据部分小于512字节，表示传输结束。对于WRQ请求，客户端发送一个小于512字节的DATA报文表示传输结束。

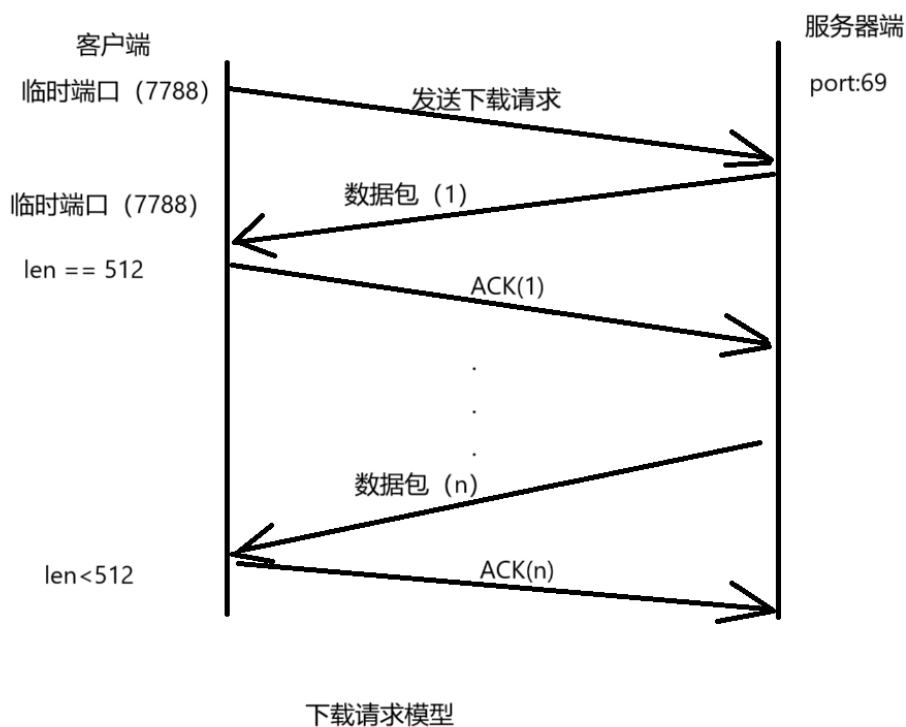
### 1.3 报文格式



有关错误码对应的信息：

- 0 未定义，差错错误信息
- 1 File Not Found
- 2 访问违规。
- 3 磁盘满或分配超出。
- 4 非法操作。
- 5 未知的传输ID。
- 6 文件已存在。
- 7 没有该用户。

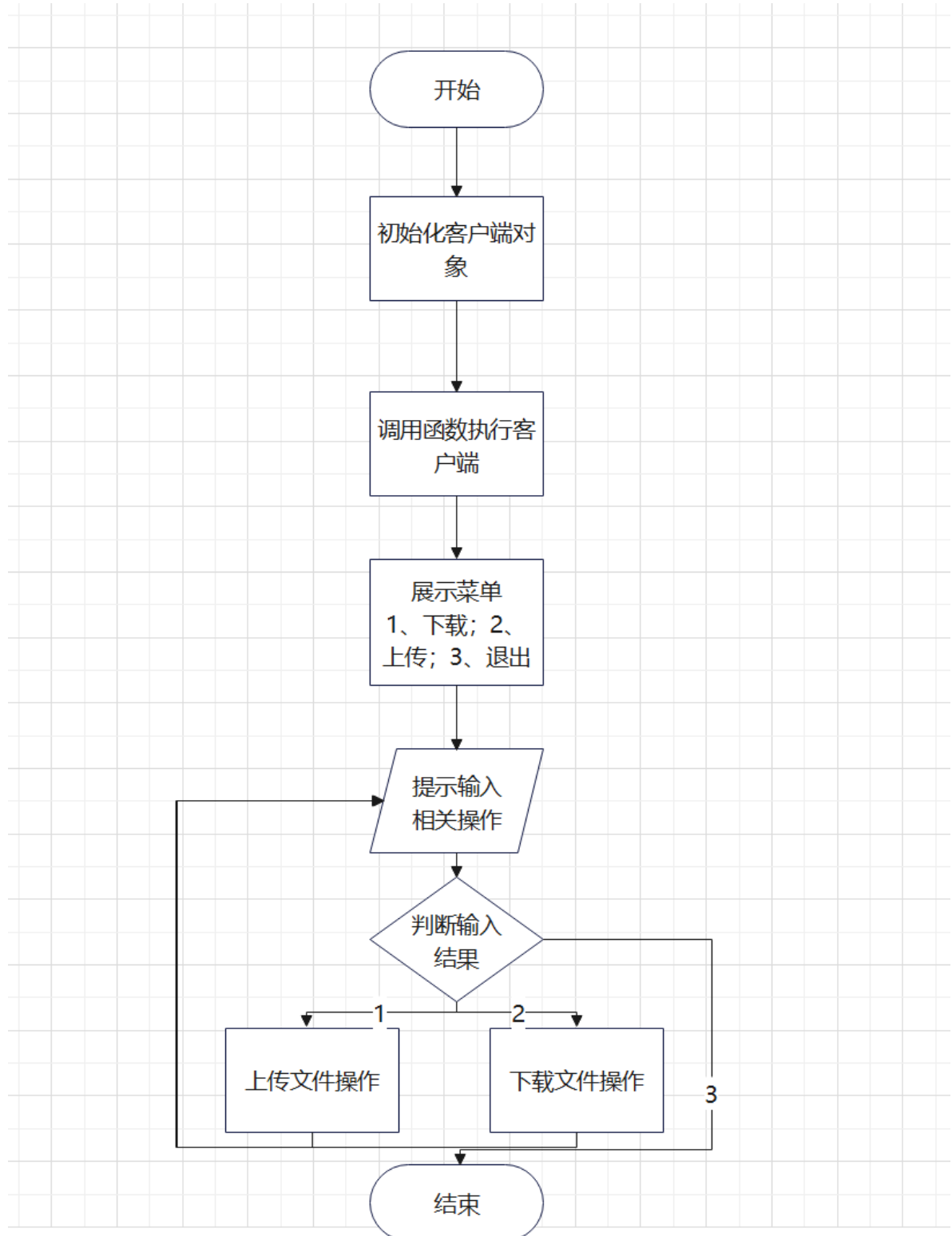
### 1.3 传输模型

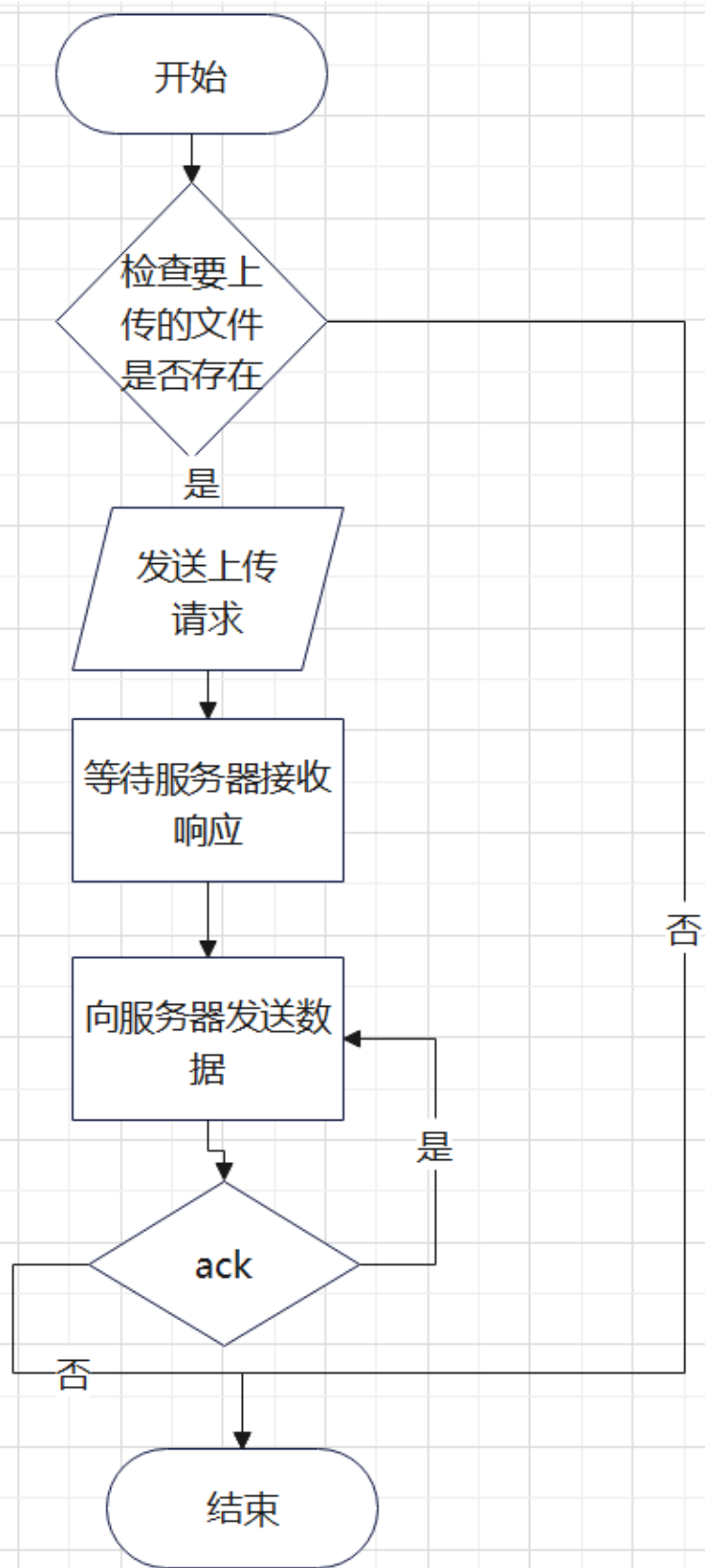


## 二、项目流程图

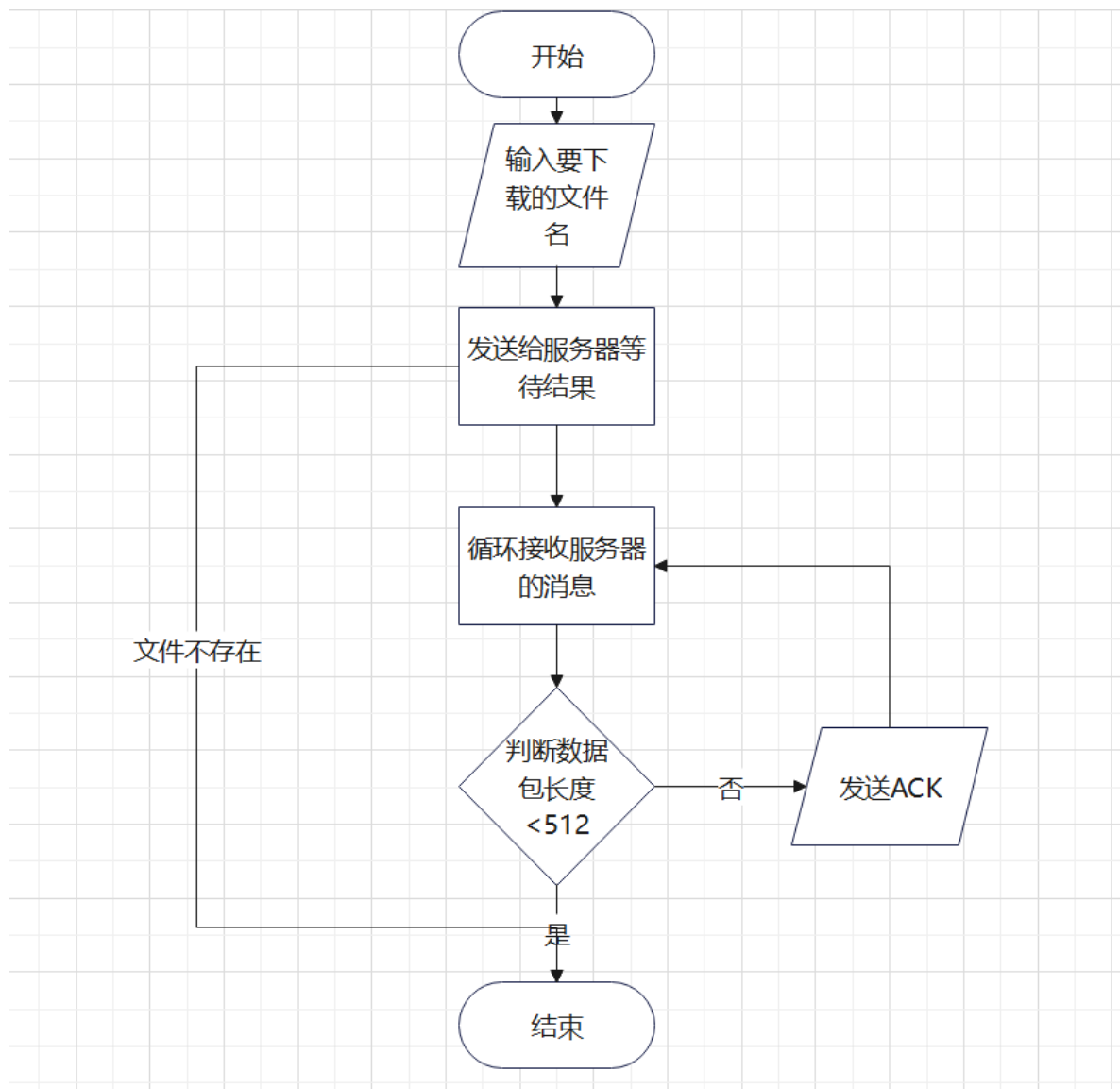
### 2.1 客户端流程图

主程序流程图





文件下载流程



## 2.2 客户端实现

### 1> 头文件

```
#ifndef TFTP_CLIENT_H
#define TFTP_CLIENT_H

//引入头文件
#include<myhead.h>
#include<errno.h>

//封装一个客户端类
class TFTPClient
{
private:
    static const int PORT = 69;          //服务器端口号
    static const int BUFFER_SIZE = 516;  //协议包的大小

    int sfd ;                             //客户端套接字
    struct sockaddr_in server_addr;       //服务器地址信息结构体

    //该客户端提供的私有的成员函数
    int doDownload() ;                    //下载函数
};
```

```

int doupload();           //上传函数
void clearScream();       //清屏函数
void waitForInput();      //等待输入函数
void showMenu();          //展示菜单函数

public:
    TFTPClient(const string & serverIP);    //构造函数
    ~TFTPClient();                          //析构函数

    void run();                             //客户端执行函数

};

#endif

```

## 2> 源文件

```

#include "tftp_client.h"

//用户打印错误信息的宏
#define ERR_LOG(msg) do{\
    perror(msg);\
    cout<<__LINE__<<" " <<__func__<<" " <<__FILE__<<endl;\
}while(0)

//构造函数的定义
TFTPClient::TFTPClient(const string & serverIP)
{
    //创建udp套接字
    this->sfd = socket(AF_INET, SOCK_DGRAM, 0);
    if(sfd < 0)
    {
        ERR_LOG("sock error");
        return;
    }

    //设置服务器地址信息结构体
    server_addr.sin_family = AF_INET;           //通信域
    server_addr.sin_port = htons(PORT);         //服务器端口号
    server_addr.sin_addr.s_addr = inet_addr(serverIP.c_str()); //服务器ip地址
}

//析构函数的定义
TFTPClient::~~TFTPClient()
{
    if(sfd > 0)
    {
        close(sfd); //如果套接字没有被关闭，则关闭套接字
    }
}

```

```

}

//定义运行函数
void TFTPClient::run()
{
    while(true)
    {
        //展示菜单
        showMenu();

        char choice;        //菜单选项
        cin >> choice;        //输入选项
        waitForInput();        //吸收垃圾字符

        switch (choice)
        {
            case '1':
                doDownload();        //执行下载请求
                break;

            case '2':
                doUpload();        //执行上传请求
                break;

            case '3':
                return;

            default:
                cout<<"您输入的功能有误，请重新输入"<<endl;
                break;
        }

        clearScream();        //清屏
    }
}

//下载函数的定义
int TFTPClient::doDownload()
{
    string filename;        //存储要下载的文件名
    cout<<"请输入要下载的文件名称: ";
    //cin >> filename;        //这种只能输入没有空格的字符串 不建议使用
    std::getline(cin, filename);        //既可以输入无空格的字符串，也可以输入有空格的字符串

    //封装下载请求
    char buf[BUFFER_SIZE] = "";
    int size = sprintf(buf, "%c%c%s%c%s%c", 0,1,filename.c_str(), 0, "octet",0);
    //封装请求包
    //向服务器发送请求
    if(sendto(sfd, buf, size, 0, (struct sockaddr*)&server_addr,
    sizeof(server_addr) )<0)
    {
        ERR_LOG("sendto error");
        return -1;
    }
}

```

```

//循环接收服务器发来的数据包
size_t recv_len;
unsigned short num = 1;           //记录接收的数据块编号
socklen_t addrlen = sizeof(server_addr); //接收服务器地址信息结构体的大小

//定义有关文件操作的遍历
int flag = 0;           //标识文件是否被打开
int fd;                 //文件描述符

//启动循环
while(true)
{
    //清空消息容器
    bzero(buf, BUFFER_SIZE);

    //读取服务器发来的消息
    recv_len = recvfrom(sfd, buf, BUFFER_SIZE, 0, (struct
sockaddr*)&server_addr, &addrlen);
    if(recv_len < 0)
    {
        ERR_LOG("recvfrom error");
        return -1;
    }

    //对读取下来的数据进行分析
    //判断是否为数据包
    if(3 == buf[1])
    {
        //读取数据操作
        //打开文件操作
        if(0 == flag) //表示文件没有被打开
        {
            fd = open(filename.c_str(), O_WRONLY|O_CREAT|O_TRUNC, 0664);
            if(fd < 0)
            {
                ERR_LOG("open error");
                return -1;
            }
            //更改标识位
            flag = 1; //表示文件已经打开
        }

        //判断数据
        if(htons(num) == *(unsigned short*)(buf+2)) //表示是传过来的数据包
        {
            //将数据包中的数据写入文件
            if(write(fd, buf+4, recv_len-4) < 0)
            {
                cout<<"fd = "<<fd<<"    recv_len = "<<recv_len<<endl;
                ERR_LOG("write error");
                close(fd);
                break;
            }
        }
    }

    //组装ACK包, 发送给服务器

```

```

        buf[1] = 4;          //ACK
        if(sendto(sfd, buf, 4, 0, (struct sockaddr*)&server_addr,
sizeof(server_addr)) < 0)
        {
            ERR_LOG("sendto error");
            close(fd);
            return -1;
        }

        //判断收取的数据长度是否小于容器长度
        if(recv_len < BUFFER_SIZE)          //表示该数据包是最后一个数据包了
        {
            cout<<"-----文件下载完毕-----"<<endl;
            close(fd);
            break;
        }

        num++;          //块编号自增

    }else if(5==buf[1])
    {
        //说明有错误信息
        cout<<"_____error: "<<(buf+4)<<"_____ "<<endl;
        if(flag == 1)          //出现错误时，文件也需要关闭
        {
            close(fd);
        }
        break;
    }
}

}

//上传函数的定义
int TFTPClient::doupload()
{
    string filename ;          //存储要上传的文件名
    cout<<"请输入要上传的文件名: ";
    getline(cin, filename);

    //检查文件是否存在
    int fd = open(filename.c_str(), O_RDONLY);          //以只读的形式打开文件，如果文件不
    存在，则报错
    if(fd < 0)
    {
        if(errno == ENOENT)
        {
            cout<<"文件不存在"<<endl;
            return -2;
        }
        ERR_LOG("open error");
        return -1;
    }
}

```

```

//程序执行至此，表示文件存在

//构造上传请求
char buf[BUFFER_SIZE] = "";
int size = sprintf(buf, "%c%c%s%c%s%c", 0, 2, filename.c_str(), 0, "octet",
0);

if(sendto(sfd, buf, size, 0, (struct sockaddr*)&server_addr,
sizeof(server_addr)) < 0)
{
    ERR_LOG("sendto error");
    close(fd);
    return -1;
}

//循环接收服务器发来的消息
int recv_len;          //接收的消息长度
unsigned short num = 0; //消息块
socklen_t addrlen = sizeof(server_addr);

while(true)
{
    //清空容器
    bzero(buf, BUFFER_SIZE);
    //读取服务器发来的消息
    recv_len = recvfrom(sfd, buf, BUFFER_SIZE, 0, (struct
sockaddr*)&server_addr, &addrlen);
    if(recv_len < 0)
    {
        ERR_LOG("recvfrom error");
        close(fd);
        return -1;
    }

    //判断消息类型
    if(4 == buf[1])
    {
        //表示收到的是服务器发来的ACK包
        //判断块编号
        if(num == ntohs(*(unsigned short *) (buf+2)))
        {
            buf[1] = 3;          //修改为数据包
            num++;                //块编号自增
            *(unsigned short *) (buf+2) = htons(num);    //修改块编号
        }

        //组装下一个数据包
        //从文件中读取数据
        int res = read(fd, buf+4, BUFFER_SIZE-4);
        if(res < 0)                //说明读取失败
        {
            ERR_LOG("read error");
            close(fd);
            break;
        }
        else if(res == 0)          //说明文件读完了
        {
            cout<<"-----上传完毕-----"<<endl;
            break;
        }
    }
}

```

```

    }

    //向服务器发送数据包
    if(sendto(sfd, buf, res+4, 0, (struct sockaddr*)&server_addr,
sizeof(server_addr))<0)
    {
        ERR_LOG("sendto error");
        close(fd);
        return -1;
    }

}

}

}

}

//清屏函数的定义
void TFTPClient::clearScream()
{
    cout<<"请输入任意字符进行清屏";
    while(getchar()!='\n');
}

//吸收垃圾字符的函数的定义
void TFTPClient::waitForInput()
{
    while(getchar()!='\n');
}

//菜单函数的定义
void TFTPClient::showMenu()
{
    //清屏
    system("clear");
    cout<<"*****基于UDP的TFTP文件传输*****"<<endl;
    cout<<"*****1、下载*****"<<endl;
    cout<<"*****2、上传*****"<<endl;
    cout<<"*****3、退出*****"<<endl;
    cout<<"*****"<<endl;
}
}

```

### 3> 主程序

```

#include"tftp_client.h"
#include<myhead.h>

int main(int argc, char const *argv[])
{
    //对输入的服务器ip地址进行判断
    if(argc < 2)

```

```

{
    cout<<"请输入IP地址"<<endl;
    return -1;
}

//实例化对象
try
{
    TFTPClient client(argv[1]); //通过有参构造，构造一个客户端
    client.run();               //执行客户端
}
catch(const std::exception& e)
{
    std::cerr << e.what() << '\n';
    return -1;
}

return 0;
}

```

#### 4> CMakeLists.txt

```

cmake_minimum_required(VERSION 2.8)

project(tftp_client)

# 设置C++标准
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

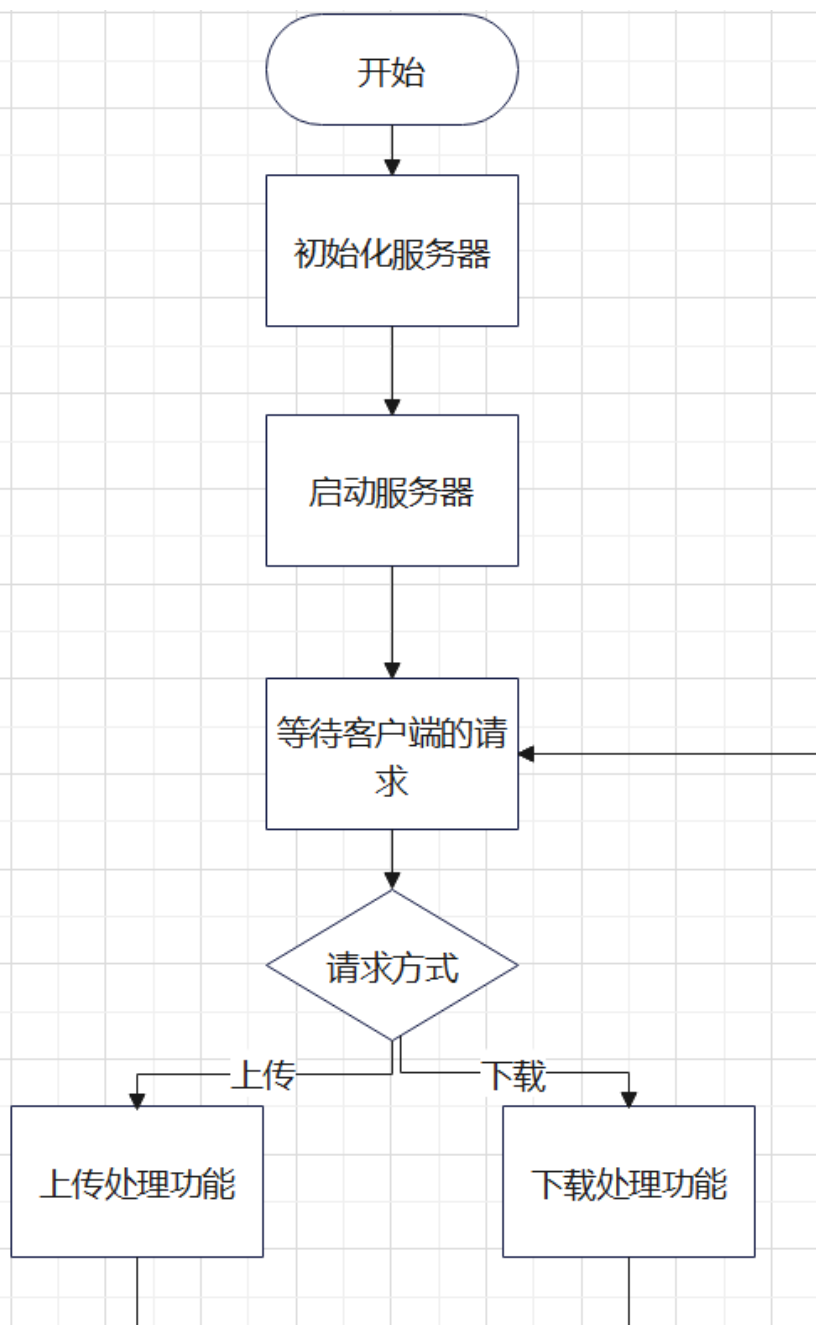
# 添加可执行程序
add_executable(${PROJECT_NAME}
    src/main.cpp
    src/tftp_client.cpp)

# 添加相关的路径
target_include_directories(${PROJECT_NAME}
    PRIVATE
    ${PROJECT_SOURCE_DIR}/header)

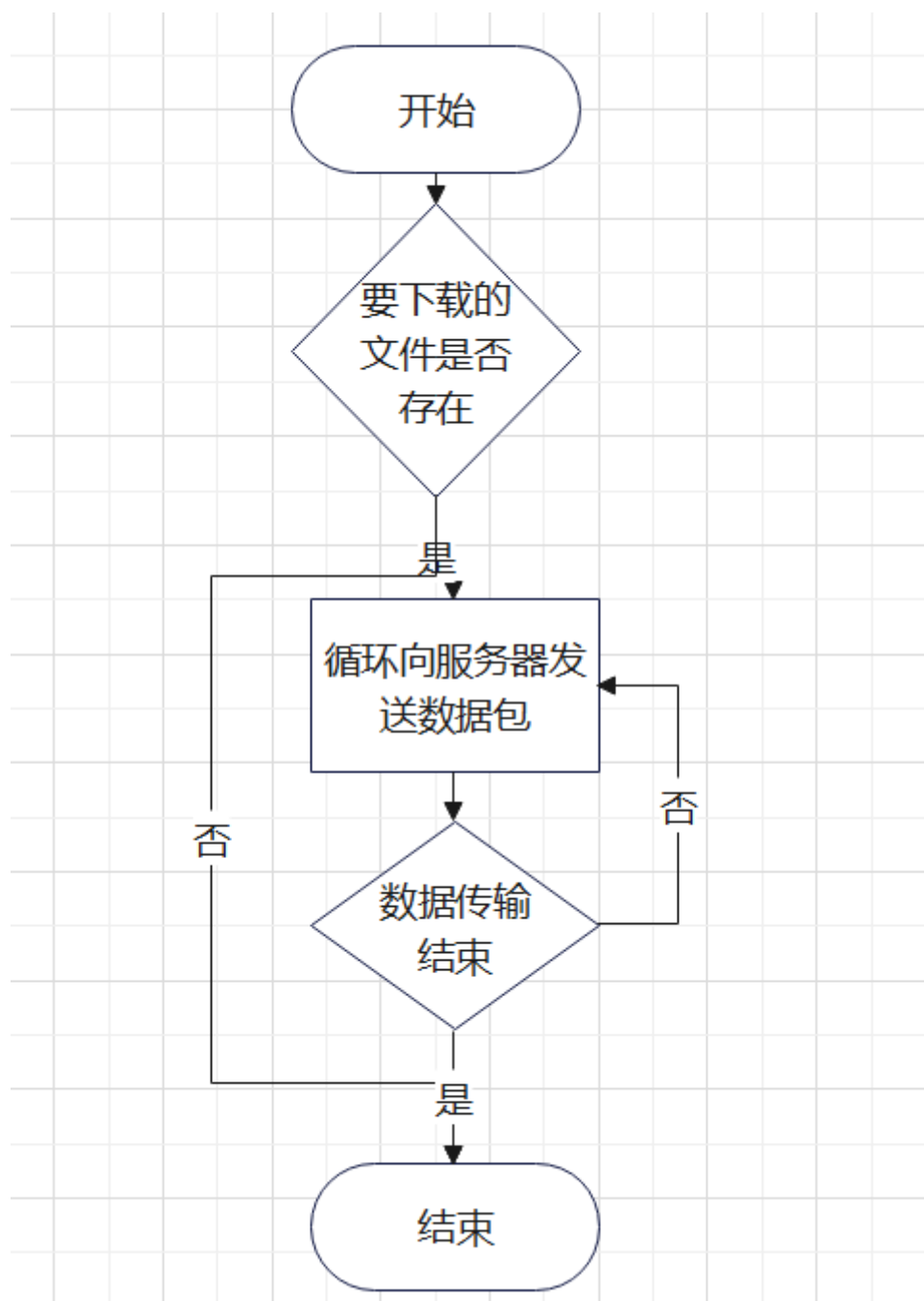
```

## 2.3 服务器端流程图

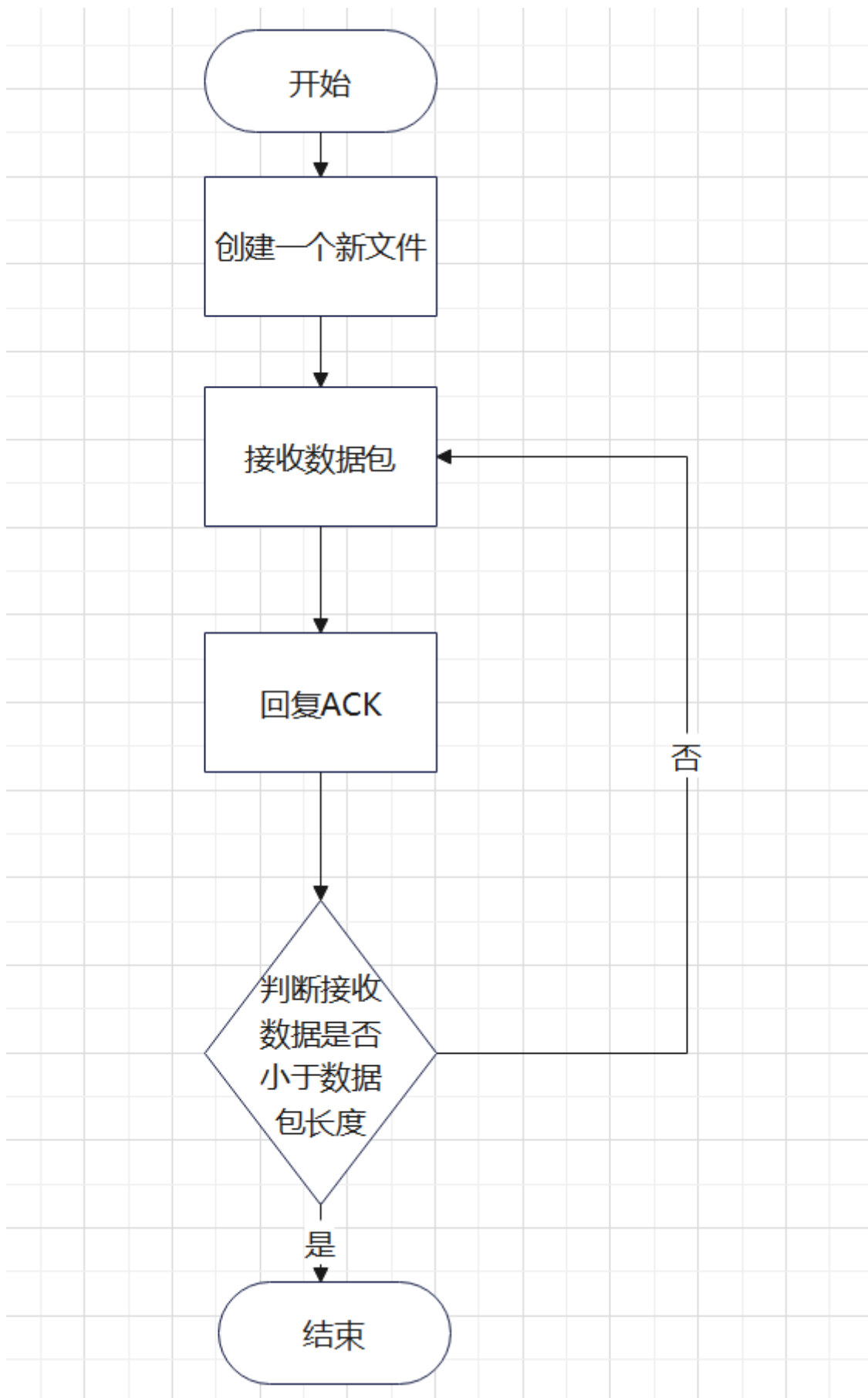
### 1> 主程序流程图



2> 下载请求流程图



3> 上传请求流程图



## 2.4 服务器端实现

1> 头文件

```
#ifndef TFTP_SERVER_H
#define TFTP_SERVER_H
```

```

#include<myhead.h>

//创建服务器类
class TFTPServer
{
private:
    static const int PORT = 6666;          //服务器的端口号
    static const int BUFFER_SIZE = 516;    //数据包的最大容量

    int sfd ;                             //服务器套接字文件描述符
    struct sockaddr_in server_addr;         //服务器地址信息结构体
    string root_dir;                       //文件服务的根目录

    //处理客户端的请求函数
    //const char *filename: 要发送给客户端的文件名
    //client_addr:客户端地址信息结构体
    //addr_len:地址信息结构体大小
    void handleReadRequest(const char *filename, struct sockaddr_in
&client_addr, socklen_t addr_len);        //处理读请求

    //char *filename: 将客户端发来的数据存储到的文件
    //client_addr:客户端地址信息结构体
    //addr_len:地址信息结构体大小
    void handleWriteRequest(const char *filename, struct sockaddr_in
&client_addr, socklen_t addr_len);        //处理写请求

    //const char * msg: 要发送给客户端的错误信息
    //client_addr:客户端地址信息结构体
    //addr_len:地址信息结构体大小
    void sendError(const char * msg, struct sockaddr_in &client_addr);

public:
    TFTPServer(const string &root = ".");    //构造函数的声明
    ~TFTPServer();                          //析构函数的声明

    void run();                             //启动服务器的函数
};

#endif

```

## 2> 源文件

```

#include"tftp_server.h"

//用户打印错误信息的宏
#define ERR_LOG(msg) do{\
    perror(msg);\
    cout<<__LINE__<<" "<<__func__<<" "<<__FILE__<<endl;\
}while(0)

//构造函数的定义
TFTPServer::TFTPServer(const string &root) :root_dir(root)

```

```

{
    //创建UDP套接字
    sfd = socket(AF_INET, SOCK_DGRAM, 0);
    if(sfd < 0)
    {
        ERR_LOG("socket error");
        return;
    }

    //填充地址信息结构体
    server_addr.sin_family = AF_INET;    //通信域
    server_addr.sin_port = htons(PORT);  //端口号
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);    //表示任意一个主机都可以向
我发消息

    //设置端口号快速重用
    int reuse = 1;
    if(setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) < 0)
    {
        ERR_LOG("setsockopt error");
        return;
    }

    //绑定工作
    if(bind(sfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0)
    {
        ERR_LOG("bind error");
        return;
    }
}

//析构函数的定义
TFTPServer::~TFTPServer()
{
    if(sfd >= 0)
    {
        close(sfd);
    }
}

//定义运行函数
void TFTPServer::run()
{
    //给出提示消息
    cout<<"TFTP Server started on port:"<<PORT <<endl;
    cout<<"Serving file from:" << root_dir<<endl;

    char buf[BUFFER_SIZE] = "";    //用来存储数据包
    struct sockaddr_in client_addr;    //用来存储客户端地址信息结构体
    socklen_t addr_len = sizeof(client_addr);    //存储客户端地址信息结构体的大小

    //循环跑服务器
    while(true)
    {
        //接收客户端的请求包
        int n = recvfrom(sfd, buf, BUFFER_SIZE, 0,
            (struct sockaddr*)&client_addr, &addr_len);
    }
}

```

```

    if(n < 0)
    {
        ERR_LOG("recvfrom error");
        return;
    }

    //检查操作码
    if(buf[0] != 0) continue;           //说明协议错误

        //解析请求包
    char * filename = buf+2;           //解析请求文件名
    const char *mode = filename + strlen(filename) +1;    //得到协议模式

    //检查传输格式
    if(strcasecmp(mode, "octet") != 0)
    {
        sendError("Only binary mode supportd", client_addr);
        continue;
    }

    //处理正常请求
    switch (buf[1])
    {
    case 1:        //读请求
        cout<<"read request for :" <<filename<<endl;
        //调用处理读请求的函数
        handleReadRequest(filename, client_addr, addr_len);
        break;

    case 2:        //写请求
        cout<<"write request for :" <<filename<<endl;
        //调用处理读请求的函数
        handleWriteRequest(filename, client_addr, addr_len);
        break;
    default:
        sendError("Unknown request", client_addr);    //处理异常请求
        break;
    }
}

}

//处理读请求的函数实现
void TFTPServer::handleReadRequest(const char *filename, struct sockaddr_in
&client_addr,

socklen_t addr_len)
{
    //找到要下载的文件路径
    string full_path = root_dir + "/" + filename;    //C++中字符串可以是+拼接
    //以只读的形式打开文件
    int fd = open(full_path.c_str(), O_RDONLY);
    if(fd < 0)
    {
        //表示要下载的文件再服务器中不存在

```

```

        sendError("File not found", client_addr);
        return;
    }

    //c程序执行至此，表示要下载的文件在服务器中存在
    char buf[BUFFER_SIZE] = "";           //数据包
    unsigned short block_num = 1;         //块编号

    //循环发送数据包
    while(true)
    {
        //准备数据包
        buf[0] = 0;
        buf[1] = 3;           //表示数据包
        *(unsigned short *)(buf+2) = htons(block_num); //块编号封装到数据包中

        //从文件中读取数据到 buf +4之后
        int n = read(fd, buf+4, BUFFER_SIZE-4);
        if(n < 0)
        {
            sendError("Read error", client_addr);
            close(fd);
            return;
        }

        //程序执行至此，表示数据包已经封装好，将数据包发送给客户端
        if(sendto(sfd, buf, n+4, 0, (struct sockaddr*)&client_addr, addr_len)<0)
        {
            ERR_LOG("sendto error");
            close(fd);
            return ;
        }

        //程序执行至此，表示服务器成功将一个数据包发送给了客户端
        //等待客户端反馈ACK
        do
        {
            //收取客户端发来的ACK数据包
            if(recvfrom(sfd, buf, BUFFER_SIZE, 0, (struct
sockaddr*)&client_addr, &addr_len) < 0)
            {
                ERR_LOG("recvfrom error");
                close(fd);
                return;
            }
        } while (buf[1]!=4 || ntohs(*(unsigned short *)(buf+2)) != block_num);

        //判断当前的文件是否读取结束
        if(n < BUFFER_SIZE-4)
        {
            break;
        }

        //程序执行至此，表示还有数据等待发送，此时块编号应该自增
        block_num++;
    }

    //关闭文件描述符

```

```

        close(fd);

    }

    //处理写请求的函数实现
    void TFTPServer::handlewriteRequest(const char *filename, struct sockaddr_in
    &client_addr,

    socklen_t addr_len)
    {
        //找到要上传的文件路径
        string full_path = root_dir + "/" + filename;    //C++中字符串可以是+拼接
        //以只写的形式打开文件
        int fd = open(full_path.c_str(), O_WRONLY|O_CREAT|O_TRUNC, 0664);
        if(fd < 0)
        {
            //表示创建文件失败
            sendError("Cannot create file", client_addr);
            return;
        }

        //程序执行至此，表示已经创建好文件
        //发送ACK
        char buf[BUFFER_SIZE] = "";    //数据包
        unsigned short block_num = 0;    //块编号

        buf[0] = 0;
        buf[1] = 4;    //ACK包
        *(unsigned short*)(buf+2) = htons(block_num);    //块编号

        //发送ACK
        if(sendto(sfd, buf, 4, 0, (struct sockaddr*)&client_addr, addr_len)<0)
        {
            ERR_LOG("sendto error");
            return ;
        }

        //循环接收数据包
        while(true)
        {
            //读取数据包
            int n = recvfrom(sfd, buf, BUFFER_SIZE, 0, (struct
            sockaddr*)&client_addr, &addr_len);
            if(n<0)
            {
                ERR_LOG("recvfrom error");
                close(fd);
                return ;
            }

            //判断是否为数据包
            if(buf[1]==3 && ntohs(*(unsigned short*)(buf+2)) == block_num+1)
            {
                //说明是数据包，将数据包中的数据，写入到文件中
                if(write(fd, buf+4, n-4) < 0)
                {
                    sendError("Write error", client_addr);
                    close(fd);
                }
            }
        }
    }
}

```

```

        return;
    }

    //向客户端发送一个ACK
    block_num++;
    buf[0] = 0;
    buf[1] = 4;    //说明是ACK包
    *(unsigned short*)(buf+2) = htons(block_num);    //块编号
    if(sendto(sfd, buf, 4, 0, (struct sockaddr*)&client_addr, addr_len)
<0)
    {
        ERR_LOG("sendto error");
        return ;
    }

    //判断读取的数据的长度跟缓冲区长度大小
    if(n < BUFFER_SIZE)
    {
        break;    //表示上传完成
    }
}

}

close(fd);    //关闭文件
}

//发送错误信息
void TFTPServer::sendError(const char * msg, struct sockaddr_in &client_addr)
{
    //封装错误包
    char buf[BUFFER_SIZE] = "";

    buf[0] = 0;
    buf[1] = 5;    //错误包的操作码
    buf[2] = 0;
    buf[3] = 1;    //错误码
    strcpy(buf+4, msg);    //错误信息

    //将消息发送给客户端
    if(sendto(sfd, buf, strlen(msg)+5, 0, (struct sockaddr*)&client_addr,
sizeof(client_addr))<0)
    {
        ERR_LOG("sendto error");
        return;
    }
}

```

### 3> 主程序

```

#include <myhead.h>
#include "tftp_server.h"    //自定义类的头文件

int main(int argc, const char *argv[])

```

```

{

    try
    {
        string root_dir = (argc>1) ? argv[1] : ".";    //如果外部传入的要操作的文件路
径，则使用操作的路径

                                                    //否则使用可执行程序所在的路径作
为要操作的路径
        //构造一个服务器
        TFTPServer server(root_dir);

        //运行服务器
        server.run();
    }
    catch(const std::exception& e)
    {
        std::cerr << e.what() << '\n';
        return -1;
    }

    return 0;
}

```

4> CMakeLists.txt

```

cmake_minimum_required(VERSION 2.8)

project(tftp_server)

# 设置C++标准
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 添加可执行程序
add_executable(${PROJECT_NAME}
    src/main.cpp
    src/tftp_server.cpp)

# 添加相关的路径
target_include_directories(${PROJECT_NAME}
    PRIVATE
    ${PROJECT_SOURCE_DIR}/header)

```

# 线程池

## 一、什么是线程池？有什么用处？

线程池是一种预先创建一组线程的机制，这些线程在应用程序启动时，就以及创建好了，等待执行线程。

当有新的任务需要执行时，线程池会从线程集合中，分配一个空闲的线程来执行该任务，而不是每一个任务都重新创建一个线程

## 二、为什么要使用线程池

### 2.1 提高性能

创建和销毁线程时开销较大的操作，通过重用现有的线程，线程池减少了这种开销，进而提高程序的性能

### 2.2 控制并发量

线程池运行你限制并发线程的数量，防止系统因创建过多的线程而出现资源耗尽的情况（如CPU过载、内存不足）

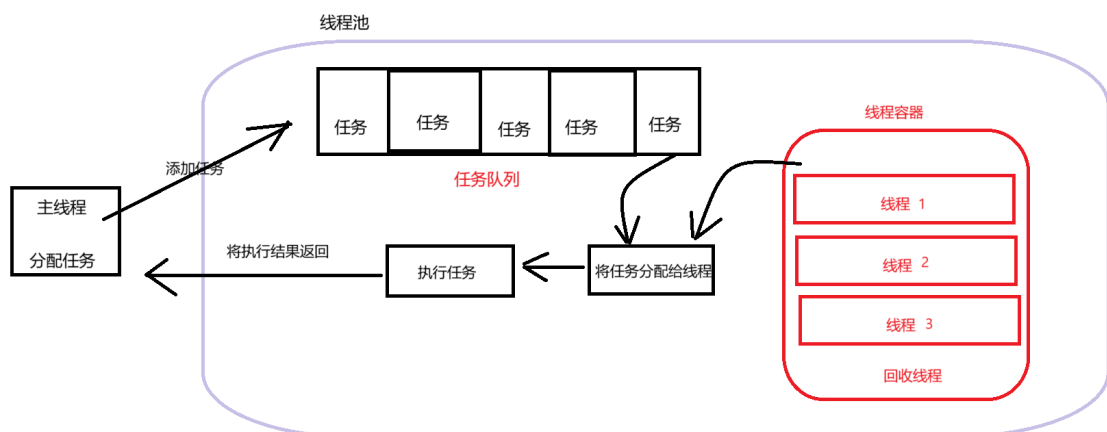
### 2.3 简化线程管理

使用线程池可以避免手动管理线程的声明周期，减少代码的复杂性。线程池通常提供了任务排队和任务调度的功能，使得多线程编程更加简化

### 2.4 使用场景

- 1> 服务器应用：对于套接字通信或者Web服务器通信
- 2> 高性能的计算：在需要进行高性能的计算时，线程池可以帮助分配和管理计算任务
- 3> 异步任务处理：在需要并行处理多个任务之间没有依赖关系时，线程池是一个理想的解决方案

## 三、线程池原理图



## 四、使用C++封装线程池的操作

- 1> 头文件

```
#ifndef THREADPOOL_H
#define THREADPOOL_H

#include<vector>           //用于存储多个线程
```

```

#include<thread>           //线程的创建及管理
#include<queue>             //用于存储多个任务的队列
#include<condition_variable> //条件变量头文件
#include<mutex>             //互斥锁头文件
#include<functional>        //用于使用函数类型
#include<myhead.h>          //自定义的头文件

class threadPool
{
private:
    //线程池相关的私有成员
    vector<thread> workers;           //存储工作的线程容器
    queue<function<void()>> tasks;    //存储任务的队列
    mutex task_mutex;                 //互斥锁
    condition_variable task_cv;       //用于通知线程有新任务的条件变量
    bool stop;                        //线程池的标志，判断线程池是否停止

    //启动线程池
    void startThreadPool(size_t numThreads);

public:
    threadPool(int threadPoolSize);    //构造函数声明
    ~threadPool();                     //析构函数声明

    void addTask(function<void()> task); //添加任务到线程池
};

#endif

```

## 2> 源文件

```

#include"threadPool.h"

//构造函数定义
threadPool::threadPool(int threadPoolSize):stop(false)
{
    startThreadPool(threadPoolSize);    //启动线程池
}

//析构函数的定义
threadPool::~~threadPool()
{
    {
        unique_lock<mutex> lock(task_mutex);    //获取锁资源，用于保护stop变量
        stop = true;
    }

    //将所有的等待线程唤醒
    task_cv.notify_all();

    //将所有的工作线程回收
    for(auto &worker : workers)
    {
        worker.join();
    }
}

```

```

    }

}

//将任务加到线程池中
void ThreadPool::addTask(function<void()> task)
{
    {
        unique_lock<mutex> lock(task_mutex);    //获取锁资源，保护条件变量
        tasks.push(task);                      //将任务添加到任务队列
    }

    //唤醒一个等待的线程开始工作
    task_cv.notify_one();    //唤醒一个线程去工作
}

//启动线程池函数的定义
void ThreadPool::startThreadPool(size_t numThreads)
{
    //循环创建numThreads个线程
    for(int i=0; i<numThreads; i++)
    {
        //创建一个线程，并且将线程放入到线程容器中
        workers.emplace_back([this]           //使用lambda表达式定义线程体
        {
            while(true)
            {
                function<void()> task;    //创建一个任务
                {
                    unique_lock<mutex> lock(task_mutex);    //加锁，保护任务队列
                    task_cv.wait(lock, [this]{    //等待条件变量通知
                        return stop || !tasks.empty(); //当线程池停止工作或者任务队列
不为空时
                    });

                    if(stop && tasks.empty())
                    {
                        return;    //当线程池停止或者任务队列为空，退出线程
                    }

                    task = move(tasks.front());    //从任务队列中取出一个任务
                    tasks.pop();    //将任务从任务队列中移除
                }
                task();    //执行任务
            }
        });
    }
}

```

### 3> 主程序

```

#include <myhead.h>
#include "ThreadPool.h"

```

```

int main(int argc, const char *argv[])
{
    threadPool mypool(3);    //一个线程池初始化4个线程

    for(int i=0; i<20; i++)    //添加20个任务到线程池
    {
        mypool.addTask([i]() {
            //使用lambda表达式当做线程体函数

            cout<<"任务: "<<i<<"正在执行, 其tid = "<<this_thread::get_id()<<endl;

            //模拟任务执行时间, 表示该任务执行100毫秒
            this_thread::sleep_for(std::chrono::milliseconds(100));

        });
    }

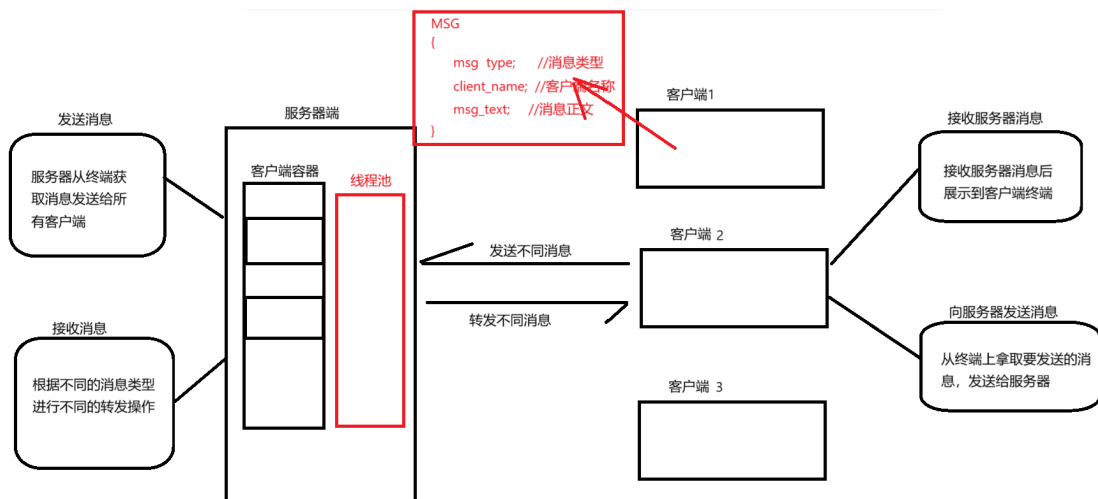
    //等待所有任务完成
    this_thread::sleep_for(std::chrono::seconds(5));

    return 0;
}

```

# 基于TCP的网络聊天室

## 一、原理图



## 二、服务器端实现逻辑

### 1> 头文件

```

#ifndef CHATSERVER_H
#define CHATSERVER_H

#include <vector>    //用于存储多个线程
#include <thread>    //线程的创建及管理
#include <queue>    //用于存储多个任务的队列

```

```

#include <condition_variable> //条件变量头文件
#include <mutex>               //互斥锁头文件
#include <functional>          //用于使用函数类型
#include <myhead.h>            //自定义的头文件

#define N 128 // 消息缓冲区的大小
#define LOGIN 1 // 表示登录消息类型
#define CHAT 2 // 表示聊天消息类型
#define QUIT 3 // 表示退出消息类型

// 自定义服务器类
class chatServer
{
public:
    // 定义消息结构体
    struct MSG
    {
        int type; // 消息类型
        char name[20]; // 客户端名称
        char text[N]; // 消息内容

        // 序列化函数：将消息结构体转为二进制数据，便于传输
        string serialize() const
        {
            string data; // 临时变量存储要转换的字符串
            data.append(reinterpret_cast<const char *>(&type), sizeof(type)); // 将消息类型放入字符串
            data.append(name, sizeof(name)); // 将消息中的客户端名称放入字符串
            data.append(text, sizeof(text)); // 将消息正文放入字符串

            return data; // 将拼接好的字符串返回
        }

        // 反序列化函数：将二进制数据转换为结构体
        void deserialize(const string &data)
        {
            size_t offset = 0; // 截取字符串时的偏移量

            memcpy(&type, data.c_str() + offset, sizeof(type)); // 将字符串的消息类型解析出来
            offset += sizeof(type); // 偏移量向后偏移

            memcpy(name, data.c_str() + offset, sizeof(name)); // 将字符串中的客户端名字解析出来
            offset += sizeof(name); // 偏移量向后偏移

            memcpy(text, data.c_str() + offset, sizeof(text)); // 将字符串中的消息正文解析出来
        }
    };

    // 客户端结构体类型
    struct Client
    {
        int fd; // 客户端套接字
    };
};

```

```

        struct sockaddr_in cin; // 客户端地址信息结构体
    };

private:
    int sfd; // 服务器套接字
    vector<Client> clients; // 在线的客户端列表
    mutex client_mutex; // 保护 clients 客户端列表的互斥锁

    // 线程池相关变量
    vector<thread> workers; // 存储工作的线程容器
    queue<function<void()>> tasks; // 存储任务的队列
    mutex task_mutex; // 互斥锁
    condition_variable task_cv; // 用于通知线程有新任务的条件变量
    bool stop; // 线程池的标志，判断线程池是否停止

    void errLog(const char *msg); // 错误信息日志函数
    void startThreadPool(size_t numThreads); // 启动线程池函数
    void addTask(function<void()> task); // 将任务添加到线程池中

public:
    chatServer(const char *ip, int port, size_t threadPoolSize = 4); // 构造函数声明
    ~chatServer(); // 析构函数声明
    void run(); // 运行服务器函数
    void handleClient(int client_fd, struct sockaddr_in cin); // 处理客户端连接的函数
    void broadcast(const MSG &msg, int exclude_fd = -1); // 广播消息函数
};

#endif

```

## 2> 源文件

```

#include "chatServer.h"

// 构造函数的定义：初始化服务器，创建套接字、绑定地址信息结构体，启动监听
chatServer::chatServer(const char *ip, int port, size_t threadPoolSize) :
stop(false)
{
    // 创建服务器套接字
    sfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sfd < 0)
    {
        perror("socket error");
        return;
    }

    // 端口号快速重用
    int reuse = 1;
    if (setsockopt(sfd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) < 0)
    {
        perror("setsockopt error");
        return;
    }
}

```

```

// 填充地址信息结构体
struct sockaddr_in sin;
sin.sin_family = AF_INET;           // 通信域  IPv4
sin.sin_port = htons(port);         // 端口号
sin.sin_addr.s_addr = inet_addr(ip); // ip地址

// 绑定地址信息结构体
if (bind(sfd, (struct sockaddr *)&sin, sizeof(sin)) < 0)
{
    perror("bind error");
    return;
}

// 启动监听
if (listen(sfd, 10) < 0)
{
    perror("listen error");
    return;
}

// 启动线程池
startThreadPool(threadPoolSize);
}

// 析构函数的定义
chatServer::~chatServer()
{
    stop = true;           // 设置停止标志，表示停止线程池
    task_cv.notify_all(); // 唤醒所有任务
    for (auto &worker : workers)
    {
        worker.join(); // 等待回收所有线程
    }

    close(sfd); // 关闭服务器套接字
}

// 启动线程池函数的定义
void chatServer::startThreadPool(size_t numThreads)
{
    // 循环创建numThreads个线程
    for (int i = 0; i < numThreads; i++)
    {
        // 创建一个线程，并且将线程放入到线程容器中
        workers.emplace_back([this] // 使用lambda表达式定义线程体
        {
            while(true)
            {
                function<void()> task; // 创建一个任务
                {
                    unique_lock<mutex> lock(task_mutex); // 加锁，保护任务队列
                    task_cv.wait(lock, [this]{ // 等待条件变量通知
                        return stop || !tasks.empty(); // 当线程池停止工作或者任务队列
不 为 空 时
                    });
                }

                if(stop && tasks.empty())

```

```

        {
            return; //当线程池停止或者任务队列为空，退出线程
        }

        task = move(tasks.front()); //从任务队列中取出一个任务
        tasks.pop(); //将任务从任务队列中移除
    }
    task(); //执行任务
} });

}

// 将任务加到线程池中
void chatServer::addTask(function<void()> task)
{
    {
        unique_lock<mutex> lock(task_mutex); // 获取锁资源，保护条件变量
        tasks.push(task); // 将任务添加到任务队列
    }

    // 唤醒一个等待的线程开始工作
    task_cv.notify_one(); // 唤醒一个线程去工作
}

// 打印错误日志函数的定义
void chatServer::errLog(const char *msg)
{
    cerr << __FILE__ << " " << __func__ << " " << __FILE__ << endl;
    perror(msg);
}

// 启动服务器函数的定义
void chatServer::run()
{
    // 循环等待客户端连接请求
    while (true)
    {
        struct sockaddr_in cin; // 用于接收客户端的地址信息结构体
        socklen_t clen = sizeof(cin); // 客户端地址信息的长度

        // 阻塞接收客户端连接请求
        int client_fd = accept(sfd, (struct sockaddr *)&cin, &clen);
        if (client_fd < 0)
        {
            errLog("accept error");
            continue; // 继续接收下一个
        }

        // 将处理客户端信息的任务，添加到线程池
        addTask([this, client_fd, cin]
        {
            //函数体就是处理客户端的功能函数
            this->handleClient(client_fd, cin); });
    }
}

// 处理客户端消息的函数定义
void chatServer::handleClient(int client_fd, struct sockaddr_in cin)

```

```

{
    // 定义一个消息类型的变量，用于接收客户端消息
    MSG msg; // 处理后的消息形态
    char buffer[sizeof(MSG)]; // 刚接收到的消息形态

    // 循环接收客户端消息
    while (true)
    {
        // 读取客户端消息
        int recv_len = recv(client_fd, buffer, sizeof(buffer), 0);

        // 当客户端下线或者recv函数错误
        if (recv_len <= 0)
        {
            // 表示客户端下线
            // 遍历客户端容器
            auto it = clients.begin(); // 定义遍历客户端容器的迭代器
            while (it != clients.end())
            {
                if (it->fd == client_fd) // 在容器中找到当前套接字
                {
                    clients.erase(it); // 将当前套接字从列表中移除
                    break;
                }
                ++it; // 指针向后偏移
            }
            close(client_fd); // 关闭套接字
            break;
        }

        // 程序执行至此，表示收到某个客户端消息，需要进行反序列化
        msg.deserialize(string(buffer, recv_len));

        // 对消息类型进行判断
        switch (ntohl(msg.type))
        {
            case LOGIN:
            {
                // 表示客户端发的是登录消息
                // 定义一个互斥锁，保护客户端容器
                unique_lock<mutex> lock(client_mutex);
                Client new_client;
                // 将新客户端的相关信息获取下来
                new_client.fd = client_fd;
                new_client.cin = cin;

                // 将新客户端放入客户端容器中
                clients.push_back(new_client);

                // 组装一条消息，广播给所有客户端
                sprintf(msg.text, "-----%s 登录成功-----", msg.name);
                broadcast(msg); // 告诉所有用户，包括自己，该用户登录成功
                break;
            }

            case CHAT:
            {

```

```

        // 表示客户端发的是聊天消息
        // 定义一个互斥锁，保护客户端容器
        unique_lock<mutex> lock(client_mutex);
        broadcast(msg, client_fd); // 除了自己，将消息转发给其他所有人

        break;
    }
    case QUIT:
    {
        // 表示客户端发的是退出消息
        // 定义一个互斥锁，保护客户端容器
        unique_lock<mutex> lock(client_mutex);
        auto it = clients.begin(); // 定义迭代器，遍历客户端容器
        while (it != clients.end())
        {
            if (it->fd == client_fd)
            {
                clients.erase(it); // 将当前用户清除容器
                break;
            }
            ++it; // 继续遍历下一个
        }
        // 将消息广播给所有人
        sprintf(msg.text, "-----%s 退出聊天室-----", msg.name);
        broadcast(msg);
        close(client_fd); // 关闭当前客户端套接字
        break;
    }
    default:
        cout << "消息类型有误" << endl;
        break;
    }
}

// 定义广播函数
void chatServer::broadcast(const MSG &msg, int exclude_fd)
{
    // 将要广播的消息进行序列化操作，转换成字符串
    string data = msg.serialize();

    // 将字符串转发给所有客户端
    for (const auto &client : clients)
    {
        if (client.fd != exclude_fd)
        {
            if (send(client.fd, data.c_str(), data.size(), 0) < 0)
            {
                perror("send error");
                return;
            }
        }
    }
}
}

```

主程序

```

#include <myhead.h>
#include "chatServer.h"

int main(int argc, const char *argv[])
{
    // 判断传参是否正确
    if (argc < 3)
    {
        cout << "请输入 ip 地址和 端口号" << endl;
        return -1;
    }

    // 传参正确，构造一个服务器
    chatServer server(argv[1], atoi(argv[2]));

    // 启动服务器
    server.run();

    return 0;
}

```

#### 4> 工程管理文件

```

cmake_minimum_required(VERSION 2.8)

project(ChatServer)      # 可执行程序名称

# 设置C++标准 允许使用C++11标准
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11")

# 添加可执行文件
add_executable(${PROJECT_NAME}
    src/main.cpp
    src/chatServer.cpp)

# 添加头文件路径
target_include_directories(${PROJECT_NAME}
    PRIVATE
    ${PROJECT_SOURCE_DIR}/header)

# 添加线程支持库
target_link_libraries(ChatServer pthread)

```

## 三、客户端实现

#### 1> 头文件

```

#ifndef CHATCLIENT_H
#define CHATCLIENT_H
#include <myhead.h>
#include <thread> //线程库

#define N 128    // 消息缓冲区的大小
#define LOGIN 1 // 表示登录消息类型

```

```

#define CHAT 2 // 表示聊天消息类型
#define QUIT 3 // 表示退出消息类型

// d定义客户端类
class ChatClient
{
    // 定义一些公共类型
public:
    // 定义消息结构体
    struct MSG
    {
        int type; // 消息类型
        char name[20]; // 客户端名称
        char text[N]; // 消息内容

        // 序列化函数：将消息结构体转为二进制数据，便于传输
        string serialize() const
        {
            string data; // 临时变量存储要转换的字符串
            data.append(reinterpret_cast<const char *>(&type), sizeof(type)); // 将消息类型放入字符串
            data.append(name, sizeof(name)); // 将消息中的客户端名称放入字符串
            data.append(text, sizeof(text)); // 将消息正文放入字符串

            return data; // 将拼接好的字符串返回
        }

        // 反序列化函数：将二进制数据转换为结构体
        void deserialize(const string &data)
        {
            size_t offset = 0; // 截取字符串时的偏移量

            memcpy(&type, data.c_str() + offset, sizeof(type)); // 将字符串的消息类型解析出来
            offset += sizeof(type); // 偏移量向后偏移

            memcpy(name, data.c_str() + offset, sizeof(name)); // 将字符串中的客户端名字解析出来
            offset += sizeof(name); // 偏移量向后偏移

            memcpy(text, data.c_str() + offset, sizeof(text)); // 将字符串中的消息正文解析出来
        }
    };

    // 类中私有成员
private:
    int cfd; // 客户端套接字
    string name; // 客户端名称
    bool running; // 判断客户端是否允许的标志

    void errLog(const char *msg); // 错误日志函数
    void sendMsg(int type, const string &text = ""); // 发送消息函数
    void recvMsg(); // 接收消息函数

```

```

public:
    ChatClient(const char *ip, int port, const string &name); // 有参构造
    ~ChatClient(); // 析构函数

    void run(); // 客户端运行函数
};

#endif

```

## 2> 源文件

```

#include "../header/chatClient.h"
// 构造函数的定义
ChatClient::ChatClient(const char *ip, int port, const string &name) :
name(name), running(true)
{
    // 创建客户端套接字
    cfd = socket(AF_INET, SOCK_STREAM, 0);
    if (cfd < 0)
    {
        perror("socket error");
        return;
    }

    // 连接服务器
    // 1、初始化地址信息结构体
    struct sockaddr_in sin;
    sin.sin_family = AF_INET; // 通信域
    sin.sin_port = htons(port); // 端口号
    sin.sin_addr.s_addr = inet_addr(ip); // 服务器ip

    // 2、连接服务器
    if (connect(cfd, (struct sockaddr *)&sin, sizeof(sin)) < 0)
    {
        perror("connect error");
        return;
    }

    // 程序执行至此，表示连接服务器成功
    // 顺便向服务器发送登录消息
    sendMsg(LOGIN);
}

// 析构函数的定义
ChatClient::~~ChatClient()
{
    sendMsg(QUIT); // 向服务器发送退出消息
    close(cfd); // 关闭套接字
}

// 定义运行客户端函数
void ChatClient::run()
{
    // 创建一个分支线程，主要用于接收服务器端的数据
    thread recvThread(&ChatClient::recvMsg, this);
}

```

```

// 主线程向服务器发送消息
string text; // 存储要发送的消息
while (running)
{
    // 从终端读取用户要发送的消息
    getline(cin, text);

    // 判断是否为退出消息
    if (text == "quit")
    {
        // 客户端退出状态
        running = false;
        // 退出消息在析构中发送
        break;
    }

    // 调用发送消息函数
    sendMsg(CHAT, text);
}

recvThread.join(); // 回收线程资源
}

// 定义输出错误日志函数
void ChatClient::errLog(const char *msg)
{
    // 输出错误的文件、功能、对应的错误行
    cerr << __FILE__ << " " << __func__ << " " << __LINE__ << endl;
    perror(msg);
}

// 定义向服务器发送消息的函数
void ChatClient::sendMsg(int type, const string &text)
{
    MSG msg; // 要发送的数据结构体
    msg.type = htonl(type); // 消息类型
    strncpy(msg.name, name.c_str(), sizeof(msg.name)); // 设置客户端用户名
    strncpy(msg.text, text.c_str(), sizeof(msg.text)); // 消息正文

    // 将要发送的消息进行序列化处理
    string data = msg.serialize();

    // 将转换后的数据发送给服务器
    if (send(cfd, data.c_str(), data.size(), 0) < 0)
    {
        errLog("send error");
        return;
    }

    cout << "消息发送成功" << endl;

    // 判断是否为退出消息
    // if(text == "quit")
    //{
    // running = false; //客户端退出状态
    // exit(EXIT_SUCCESS); //退出进程
    //}

```

```

}

// 定义接收服务器发来消息的函数
void ChatClient::recvMsg()
{
    // 用于接收消息
    char buffer[sizeof(MSG)];
    while (running)
    {
        // 非阻塞接收客户端消息
        int recv_len = recv(cfd, buffer, sizeof(buffer), MSG_DONTWAIT);
        if (recv_len < 0)
        {
            if (errno == EAGAIN)
            {
                // 休眠100ms
                usleep(100000);
            }
            else
            {
                // 错误
                errLog("recv error");
                running = false;
                break;
            }
        }
        else if (recv_len == 0)
        {
            // 错误
            errLog("recv error");
            running = false;
            break; // 连接关闭
        }
        else
        {
            // 将消息进行解析
            MSG msg;
            // 反序列化
            msg.deserialize(string(buffer, recv_len));
            cout << msg.name << ": " << msg.text << endl;
        }
    }
}

```

### 3> 主程序

```

#include "../header/chatClient.h"

#include <myhead.h>

int main(int argc, const char *argv[])
{
    // 确保执行时，外部传入了ip地址、端口号、用户名
    if (argc < 4)

```

```

{
    cout << "请输入 ip 端口号 用户名" << endl;
    return -1;
}

// 实例化一个客户端
ChatClient client(argv[1], atoi(argv[2]), argv[3]);

// 运行客户端
client.run();

return 0;
}

```

#### 4> 工程管理文件

```

cmake_minimum_required(VERSION 2.8)

project(ChatClient)      # 可执行程序名称

# 设置C++标准 允许使用C++11标准
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11")

# 添加可执行文件
add_executable(${PROJECT_NAME}
    src/main.cpp
    src/chatClient.cpp)

# 添加头文件路径
target_include_directories(${PROJECT_NAME}
    PRIVATE
    ${PROJECT_SOURCE_DIR}/header)

# 添加线程支持库
target_link_libraries(ChatClient pthread)

```

# 基于TCP的网络电子词典

## 一、sqlite3数据库

### 1.1 数据库的概念

- 1> 数据：能够被计算机识别、存储、处理的用于描述客观信息的符号就称为数据
- 2> 数据库：是在数据库管理系统管理和控制之下，存放在存储介质上的数据的集合称为数据库
 

对于程序而言，数据库本质上可以理解成外部文件操作
- 3> 记录：能够描述一条数据的完整信息称为一条记录，例如一个学生的完整信息
- 4> 字段：能够描述数据的不可分割的最小单位，例如学生信息中的性别、身份证号
- 5> 常用的数据库：

- 1、Oracle数据库：用于大型的数据库管理
- 2、mysql数据库：用于小型的关系数据库管理
- 3、sqlserver：也是用于小型的关系数据库
- 4、selite：主要用于嵌入式linux开发的轻量级的数据库

## 1.2 sqlite3数据库的基础

1> 官网：<https://sqlite.org/>

什么是SQLite?

SQLite是一个c语言库，它实现了一个小型、快速、自包含、高可靠性、全功能的SQL数据库引擎。SQLite是世界上使用最多的数据库引擎。SQLite内置在所有手机和大多数电脑中，并捆绑在人们每天使用的无数其他应用程序中。更多信息...

SQLite文件格式是稳定的、跨平台的、向后兼容的，开发人员承诺到2050年都将保持这种状态。SQLite数据库文件通常用作在系统[1][2][3]之间传输丰富内容的容器，并作为数据[4]的长期归档格式。目前有超过1万亿（1e12）个SQLite数据库在活跃使用。

SQLite源代码属于公共领域，任何人都可以免费用于任何目的。

2> 一般linux系统是支持sqlite3数据库的，但是不一定安装头文件库了，想要安装sqlite3的头文件库

```
sudo yum install sqlite-devel
```

验证是否装了sqlite3数据的头文件：`ls /usr/include/sqlite3.h`

如果不显示没有该文件，则说明，头文件库已经安装好了

## 1.3 数据库的常规操作

学习资料网址：<https://www.runoob.com/sqlite/sqlite-tutorial.html>

1> 相关指令：在终端上直接可以执行的

- 1、创建数据库指令：**sqlite3 数据库名字** 即可创建/打开一个数据库
- 2、操作数据库分为两种：点命令和sql语句，点命令是以点（.）开头的，没有分号结尾的，sql语句是没有点开头，必须以分号结尾
- 3、点命令

上面的命令会显示小窗口里安装的 SQLite 点命令的列表，如下所示。

| 命令                    | 描述                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .backup ?DB? FILE     | 备份 DB 数据库（默认是 "main"）到 FILE 文件。                                                                                                                                                                                                                                                                                                                              |
| .bail ON OFF          | 发生错误后停止。默认为 OFF。                                                                                                                                                                                                                                                                                                                                             |
| .databases            | 列出数据库的名称及其所依附的文件。                                                                                                                                                                                                                                                                                                                                            |
| .dump ?TABLE?         | 以 SQL 文本格式转储数据库。如果指定了 TABLE 表，则只转储匹配 LIKE 模式的 TABLE 表。                                                                                                                                                                                                                                                                                                       |
| .echo ON OFF          | 开启或关闭 echo 命令。                                                                                                                                                                                                                                                                                                                                               |
| .exit                 | 退出 SQLite 提示符。                                                                                                                                                                                                                                                                                                                                               |
| .explain ON OFF       | 开启或关闭适合于 EXPLAIN 的输出模式。如果没有带参数，则为 EXPLAIN on，即开启 EXPLAIN。                                                                                                                                                                                                                                                                                                    |
| .header(s) ON OFF     | 开启或关闭头部显示。                                                                                                                                                                                                                                                                                                                                                   |
| .help                 | 显示消息。                                                                                                                                                                                                                                                                                                                                                        |
| .import FILE TABLE    | 导入来自 FILE 文件的数据到 TABLE 表中。                                                                                                                                                                                                                                                                                                                                   |
| .indices ?TABLE?      | 显示所有索引的名称。如果指定了 TABLE 表，则只显示匹配 LIKE 模式的 TABLE 表的索引。                                                                                                                                                                                                                                                                                                          |
| .load FILE ?ENTRY?    | 加载一个扩展库。                                                                                                                                                                                                                                                                                                                                                     |
| .log FILE off         | 开启或关闭日志。FILE 文件可以是 stderr（标准错误）/stdout（标准输出）。                                                                                                                                                                                                                                                                                                                |
| .mode MODE            | 设置输出模式，MODE 可以是下列之一： <ul style="list-style-type: none"><li>• <b>csv</b> 逗号分隔的值</li><li>• <b>column</b> 左对齐的列</li><li>• <b>html</b> HTML 的 &lt;table&gt; 代码</li><li>• <b>insert</b> TABLE 表的 SQL 插入（insert）语句</li><li>• <b>line</b> 每行一个值</li><li>• <b>list</b> 由 .separator 字符串分隔的值</li><li>• <b>tabs</b> 由 Tab 分隔的值</li><li>• <b>tcl</b> TCL 列表元素</li></ul> |
| .nullvalue STRING     | 在 NULL 值的地方输出 STRING 字符串。                                                                                                                                                                                                                                                                                                                                    |
| .output FILENAME      | 发送输出到 FILENAME 文件。                                                                                                                                                                                                                                                                                                                                           |
| .output stdout        | 发送输出到屏幕。                                                                                                                                                                                                                                                                                                                                                     |
| .print STRING...      | 逐字地输出 STRING 字符串。                                                                                                                                                                                                                                                                                                                                            |
| .prompt MAIN CONTINUE | 替换标准提示符。                                                                                                                                                                                                                                                                                                                                                     |
| .quit                 | 退出 SQLite 提示符。                                                                                                                                                                                                                                                                                                                                               |
| .read FILENAME        | 执行 FILENAME 文件中的 SQL。                                                                                                                                                                                                                                                                                                                                        |
| .schema ?TABLE?       | 显示 CREATE 语句。如果指定了 TABLE 表，则只显示匹配 LIKE 模式的 TABLE 表。                                                                                                                                                                                                                                                                                                          |
| .separator STRING     | 改变输出模式和 .import 所使用的分隔符。                                                                                                                                                                                                                                                                                                                                     |
| .show                 | 显示各种设置的当前值。                                                                                                                                                                                                                                                                                                                                                  |
| .stats ON OFF         | 开启或关闭统计。                                                                                                                                                                                                                                                                                                                                                     |

|                   |                     |
|-------------------|---------------------|
| .tables ?PATTERN? | 列出匹配 LIKE 模式的表的名称。  |
| .timeout MS       | 尝试打开锁定的表 MS 毫秒。     |
| .width NUM NUM    | 为 "column" 模式设置列宽度。 |
| .timer ON OFF     | 开启或关闭 CPU 定时器。      |

让我们尝试使用 **.show** 命令，来查看 SQLite 命令提示符的默认设置。

```
sqlite>.show
echo: off
```

常用的点命令：

.table :能够展示当前数据库中有哪些数据表  
.quit : 退出数据库操作的命令行  
.schema :查看数据表的结构  
.header on:展示数据信息时加表头  
.mode column: 以列的形式左对齐

2> sql语句：专门用于对数据库进行增删改查操作的语法结构称为sql语句，所有的sql语句以分号结束

### 1、创建数据表

- 1、 **create table** 表名 (字段名1 类型1, 字段名2 类型2, ... , 字段名n 类型n); //创建一个数据表
- 2、 **CREATE TABLE IF NOT EXISTS** 表名 (字段名1 类型1, 字段名2 类型2, ... , 字段名n 类型n); //即使创建同名的表不会报错

```
sqlite> CREATE TABLE Student (numb int, name text, sex char, score double);
sqlite> CREATE TABLE Student (numb int, name text, sex char, score double, telephone text);
Error: table Student already exists
sqlite> CREATE TABLE IF NOT EXISTS Student (numb int, name text, sex char, score double, telephone text);
sqlite> .table
Student
sqlite> .schema
CREATE TABLE Student (numb int, name text, sex char, score double);
sqlite>
```

创建一个数据表

展示当前数据库中的所有数据表

展示数据表的结构

如果数据表不存在则创建，否则不创建

### 2、删除数据表

**DROP TABLE** database\_name.table\_name;

### 3、插入记录：有两种操作

- 1、 **INSERT INTO** TABLE\_NAME [(column1, column2, column3,...columnN)] **VALUES** (value1, value2,value3,...valueN); //可以选择指定字段添加
- 2、 **INSERT INTO** TABLE\_NAME **VALUES** (value1,value2,value3,...valueN); //全部字段添加

```

Student
sqlite> .schema
CREATE TABLE Student (numb int, name text, sex char, score double, telephone text);
sqlite> INSERT INTO Student VALUES (1001, "张三", "男", 99.9, "13999999999");
sqlite> INSERT INTO Student (numb, name, telephone) VALUES (1002, "李四", "13888888888");
sqlite>

```

全字段添加  
指定字段添加

#### 4、查询记录:

```

SELECT column1, column2, columnN FROM table_name;    //通过指定字段查找记录
SELECT * FROM table_name;                            //查询所有信息

```

```

sqlite> select * FROM Student ;
1001|张三|男|99.9|13999999999
1002|李四|||13888888888
sqlite> .head
Usage: headers on|off
sqlite> .headers on
sqlite> select * FROM Student ;
numb|name|sex|score|telephone
1001|张三|男|99.9|13999999999
1002|李四|||13888888888
sqlite> .mode column
sqlite> select * FROM Student ;
numb      name      sex      score      telephone
-----
1001      张三      男        99.9      13999999999
1002      李四

```

展示表头  
列对齐展示信息  
将数据表中的所有信息查询

#### 5、修改记录

```

UPDATE table_name SET column1 = value1, column2 = value2..., columnN = valueN
WHERE [condition];

```

```

sqlite> .mode column
sqlite> select * FROM Student ;
numb      name      sex      score      telephone
-----
1001      张三      男        99.9      13999999999
1002      李四
sqlite> UPDATE Student set sex="女",score=88.88 WHERE numb==1002;
sqlite> select * FROM Student ;
numb      name      sex      score      telephone
-----
1001      张三      男        99.9      13999999999
1002      李四      女        88.88     13888888888
sqlite>

```

修改记录信息

#### 6、删除记录

```

DELETE FROM table_name WHERE [condition];

```

```

sqlite> DELETE FROM Student WHERE name="张三";
sqlite> select * FROM Student ;
numb      name      sex      score      telephone
-----
1002      李四      女        88.88     13888888888
sqlite>

```

删除数据表中的记录

## 1.4 sqlite3数据库编程

在程序中对数据库操作，无外乎三种：打开数据库、执行sql语句、关闭数据库，想要使用这些函数，需要链接sqlite3库，-lsqlite3

### 1> 打开数据库函数

```
int sqlite3_open(  
    const char *filename,      /* Database filename (UTF-8) */  
    sqlite3 **ppDb             /* OUT: SQLite db handle */  
);  
  
/**  
功能：打开或者创建一个数据库并打开  
参数1：要打开的数据库名称，是一个字符串  
参数2：返回打开的数据库句柄，是数据库的指针。如果没有内存分配sqlite3对象，那么会返回NULL  
返回值：成功创建或打开数据库返回SQLITE_OK(0)，失败返回错误码（注意不是置位错误码）  
*/
```

### 2> 处理错误信息的函数

```
int sqlite3_errcode(sqlite3 *db);    //返回数据库句柄操作中错误码的值  
const char *sqlite3_errmsg(sqlite3*); //返回数据库句柄操作中的错误信息
```

### 3> 关闭数据库

```
int sqlite3_close(sqlite3*);  
  
/**  
功能：关闭数据库句柄指针操作的数据库  
参数：数据库句柄指针  
返回值：成功返回SQLITE_OK，失败返回错误码  
*/
```

### 4> 执行sql语句的函数

```
int sqlite3_exec(  
    sqlite3*,                      /* 已经打开的数据库句柄指针 */  
    const char *sql,               /* 要被执行的sql语句 */  
    int (*callback)(void*,int,char**,char**), /* 回调函数 */  
    void *,                       /* 回调函数的第一个参数 */  
    char **errmsg                  /* 错误信息 */  
);  
  
/**  
功能：在指定的数据库中执行给定的sql语句  
返回值：成功返回SQLITE_OK失败返回错误码  
  
有关该函数的第三个参数：是一个函数指针，需要传递一个功能函数  
    如果sql语句是执行数据库的增删改操作，那么第三个参数填NULL即可  
    如果sql语句是进行查找操作，那么必须传递一个功能参数，查找的结果集中的每个结果都会执行该函数  
*/
```

使用C语言实现一个简单的学生管理系统，用于练习上述函数

```
#include <myhead.h>

//定义添加学生信息函数
void do_add(sqlite3* ppDb)
{
    //准备sql语句
    int add_numb = 0;    //要添加的学生学号
    char add_name[20] = "";    //要添加的学生姓名
    double add_score = 0;    //要添加的学生的成绩

    printf("请输入学生的学号: ");
    scanf("%d", &add_numb);
    printf("请输入学生的姓名: ");
    scanf("%s", add_name);
    printf("请输入学生的成绩: ");
    scanf("%lf", &add_score);
    getchar();

    //数据已经有了，需要将其转成字符串
    char sql[128] = "";    //存储要执行的sql语句
    sprintf(sql, "insert into STUDENT values(%d, \"%s\", %.2lf);", add_numb,
add_name, add_score);
    char *errmsg = NULL;

    //执行sql语句
    if(sqlite3_exec(ppDb, sql, NULL, NULL, &errmsg) != SQLITE_OK)
    {
        printf("添加信息失败,失败原因: %s\n", errmsg);
        sqlite3_free(errmsg);
        return ;
    }

    printf("添加成功\n");
}

//定义回调函数，处理查找数据后的结果集
int callback(void* arg,int cols, char** msgText,char** headText)
{
    // numb    name    score
    // 1001    zpp    99.0
    //输出表头
    if(*((int *)arg) == 0)
    {
        for(int i=0; i<cols; i++)
        {
            printf("%s\t", *(headText+i));
        }
        printf("\n");
        *((int *)arg) = 1;
    }

    //输出该结果的记录信息
    for(int i=0; i<cols; i++)
    {
        printf("%s\t", *(msgText+i));
    }
}
```

```

    }
    printf("\n");

    return 0;
}

//定义查找函数
void do_search(sqlite3 *ppDb)
{
    //准备sql语句
    const char *sql = "select * from STUDENT;";
    char *errmsg = NULL;

    int flag = 0;        //标识位

    //执行sql语句
    if(sqlite3_exec(ppDb, sql, callback, &flag, &errmsg) != SQLITE_OK)
    {
        printf("查找失败,失败原因: %s\n", errmsg);
        sqlite3_free(errmsg);
        return ;
    }
}

/*****主程序 *****/
int main(int argc, const char *argv[])
{
    //定义数据库指针
    sqlite3 *ppDb = NULL;
    //打开数据库
    if(sqlite3_open("./student.db", &ppDb) != SQLITE_OK)
    {
        printf("数据库打开失败,错误码为: %d, 错误信息为: %s\n", sqlite3_errcode(ppDb),
        sqlite3_errmsg(ppDb));
        return -1;
    }

    printf("sqlite3_open success\n");
    //程序执行至此,表示数据库已经打开

    //创建数据表
    //准备sql语句
    const char *sql = "create table if not exists STUDENT(num int, name text,
score double);";
    char *errmsg = NULL;        //用于接收执行错误语句后的错误信息
    if(sqlite3_exec(ppDb, sql, NULL, NULL, &errmsg) != SQLITE_OK)
    {
        printf("数据表创建失败,失败原因: %s\n", errmsg);
        sqlite3_free(errmsg);
        return -1;
    }
    printf("数据表创建成功\n");

    //程序执行至此,表示数据表创建成功

```

```

//做功能管理菜单
int menu = -1;
while(1)
{
    printf("\t\t-----XXX 学生管理系统-----\n");
    printf("\t\t-----1、添加学生信息-----\n");
    printf("\t\t-----2、删除学生信息-----\n");
    printf("\t\t-----3、修改学生信息-----\n");
    printf("\t\t-----4、查找学生信息-----\n");
    printf("\t\t-----0、退出-----\n");
    printf("请输入功能: ");
    scanf("%d", &menu);
    getchar();

    //对功能选项多分枝选择
    switch(menu)
    {
        case 1:
        {
            do_add(ppDb);          //执行添加数据函数
        }
        break;

        case 2:
        {
            //do_delete(ppDb);      //执行删除数据函数
        }
        break;

        case 3:
        {
            //do_update(ppDb);      //执行修改数据函数
        }
        break;

        case 4:
        {
            do_search(ppDb);        //执行查找数据函数
        }
        break;

        case 0:exit(EXIT_SUCCESS);
        default:printf("您输入的功能有误, 请重新输入\n");

    }

}

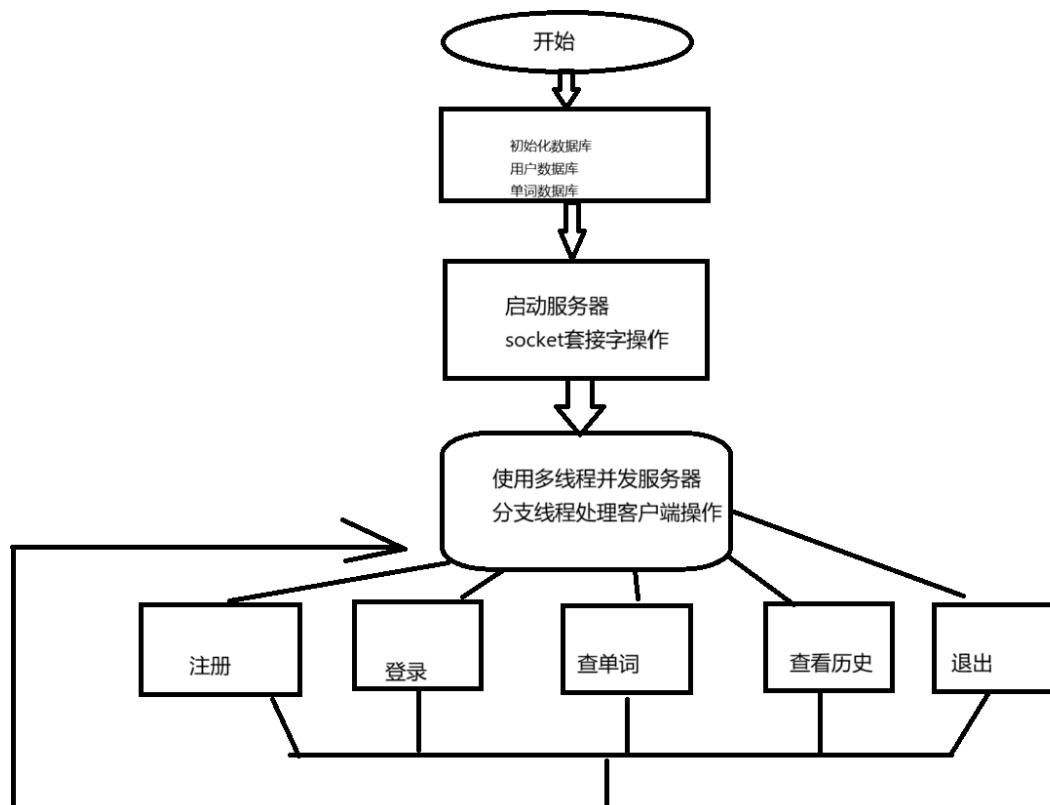
return 0;
}

```

作业：完善学生管理系统，将上面程序代码中的删除和修改功能完善

## 二、网络电子词典服务器端实现

## 2.1 流程设计



## 2.2 主要模块

1> 数据库模块：两个数据库，用户数据库、单词数据库

单词数据库：存储四级单词词汇表

用户数据库：两个表：

1、存储用户名和用户的密码

2、用户查询历史记录表

2> 服务器模块：创建服务器套接字、绑定IP和端口号、启动监听、接收连接请求、分线程管理客户端

3> 数据模块：

1、传输数据类型：

用户注册类型、用户登录类型、用户退出类型、单词查询类型、查询历史记录类型

2、数据类型：

该数据传输的类型

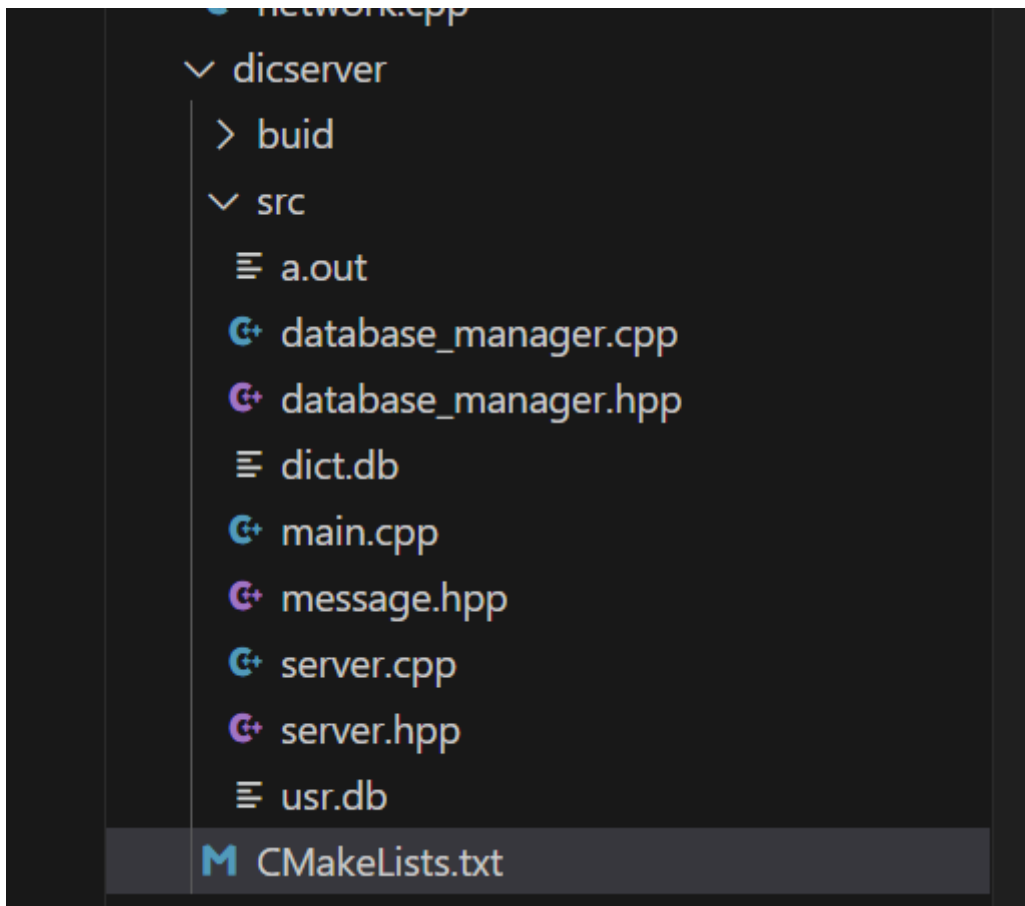
用户名

传输文本

注意：也要进行数据转换：网络字节序和主机字节序的转换

## 2.3 代码实现

项目结构：



1> database\_manager.cpp

```
#include "database_manager.hpp"    //包含自定义的头文件
#include <fstream>                 //IO操作的头文件
#include <ctime>                   //时间的头文件
#include <stdexcept>               //异常处理头文件

#define DICT_PATH "./dict.txt"    //单词文本的文件路径

/*
    构造函数：打开数据库连接
    参数1：用户数据库的文件路径
    参数2：词典数据库的文件路径
    返回值：如果数据库打开失败，则返回异常处理：runtime_error
*/
DatabaseManager::DatabaseManager(const string &usr_db_path, const string
&dict_db_path)
{
    //打开或创建用户数据库
    if(sqlite3_open(usr_db_path.c_str(), &usr_db_) != SQLITE_OK)
    {
        throw runtime_error("用户数据库打开失败: " +
string(sqlite3_errmsg(usr_db_)));    //抛出异常
    }

    //打开或创建单词数据库
    if(sqlite3_open(dict_db_path.c_str(), &dict_db_) != SQLITE_OK)
    {
        throw runtime_error("单词数据库打开失败: " +
string(sqlite3_errmsg(dict_db_)));    //抛出异常
    }
}
```

```

}

/*
    析构函数：关闭数据库的连接
*/

DatabaseManager::~DatabaseManager()
{
    sqlite3_close(usr_db_);          //关闭用户数据库
    sqlite3_close(dict_db_);         //关闭单词数据库
}

/*
    初始化数据库：创建相关数据表，并且导入单词库
*/
bool DatabaseManager::initilizeDatabase()
{
    return initializeUserDB() && initializeDictDB();    //用户表和单词表都成功
    后，表示数据库初始化成功
}

/*
    初始化用户表：创建一个用户表
*/

bool DatabaseManager::initializeUserDB()
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句：创建用户表和历史记录表的sql语句
    const char *sql = "create table if not exists usr("    //创建用户表
                      "name text primary key,"          //用户名（主键）
                      "passwd int,"                     //密码
                      "stage int);"                     //用户状态：0表示离线    1
    表示在线

                      "create table if not exists history(" //创建历史记录表
                      "name text,"                        //用户名
                      "word text,"                         //查询的单词
                      "mean text,"                        //单词意思
                      "time text);"                       //查询时间

    //执行sql语句
    if(!executesQL(usr_db_, sql))
    {
        cerr<<"用户数据表初始化失败"<<endl;
        return false;
    }

    //重置所有的客户端都为离线状态
    sql = "update usr set stage=0;";
    if(!executesQL(usr_db_, sql))
    {
        cerr<<"用户状态重置失败"<<endl;
        return false;
    }
}

```

```

        return true;                //表示两个数据表创建成功，并且将所有客户端设置成离线状态
    }

    /*
        初始化单词数据库的表结构，并且进行单词的导入
    */
    bool DatabaseManager::initializedDictDB()
    {
        //创建互斥锁，保护单词表
        lock_guard<mutex> lock(dict_mutex_);

        //准备创建单词数据库的sql语句
        const char *sql = "create table if not exists dict(word text, mean text);";
        if(!executesQL(dict_db_, sql))
        {
            cerr<<"单词表创建失败"<<endl;
            return false;
        }

        //检查词典数据库是否为空
        sql = "select count(*) from dict;";          //查询数据库中的记录个数
        sqlite3_stmt *stmt;                          //接收预处理sql的结果
        //将上面的sql预编译一下
        //函数介绍：将sql语句进行预编译一遍，用于后面的执行
        //参数1：数据库句柄
        //参数2：要编译的sql语句
        //参数3：-1表示sql是一个字符串
        //参数4：编译后的结果存放的位置
        //参数5：接收没有处理的剩余的sql语句
        if(sqlite3_prepare_v2(dict_db_, sql, -1, &stmt, NULL) != SQLITE_OK)
        {
            cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
            return false;
        }

        bool need_import = false;                    //标识是否需要导入数据

        //执行sql语句
        if(sqlite3_step(stmt) == SQLITE_ROW)          //表示成功执行
        {
            if(sqlite3_column_int(stmt, 0) == 0)      //表示数据为空
            {
                //表示表为空，需要导入数据
                need_import = true;
            }
        }

        //释放stmt空间
        //功能：是否预编译sql语句的空间
        sqlite3_finalize(stmt);

        //如果需要导入，返回导入函数的结果，如果已经有了数据表内容，则直接返回
        return need_import ? dictToDatabase() : true;
    }

```

```

/*
将单词文件中的信息导入到数据库中
*/

bool DatabaseManager::dictToDatabase()
{
    //使用文件操作，打开文件dict.txt
    ifstream file(DICT_PATH);
    if(!file)
    {
        cerr<<"无法打开词典文件:"<<DICT_PATH<<endl;
        return false;
    }

    string line;          //用于读取文件中的一行数据
    while(getline(file,line)) //循环从文件中读取一行信息放入到line中
    {
        //跳过空行
        if(line.empty()) continue;

        //分割单词和意思 （格式：单词 意思）
        size_t pos = line.find(' '); //定位到空格所在的位置
        if(pos == string::npos) //表示没有该位置，也就是说该行没有空格
        {
            cerr<<"无效的词典条目（缺少空格分隔）"<<line<<endl;
            continue;
        }

        //将单词和含义分别存储
        string word = line.substr(0, pos); //将当前行中的单词截取出来
        string mean = line.substr(pos+1); //将当前行中的单词含义截取出来

        //准备sql语句 可以使用通配符表示
        const char *sql = "insert into dict values(?,?)";
        //预编译sql语句
        sqlite3_stmt * stmt = NULL;
        if(sqlite3_prepare_v2(dict_db_, sql, -1, &stmt, NULL) !=SQLITE_OK)
        {
            cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
            file.close();
            return false;
        }

        //预编译成功，可以给已经预编译成功的内容绑定参数
        sqlite3_bind_text(stmt, 1, word.c_str(), -1, SQLITE_STATIC);
        sqlite3_bind_text(stmt, 2, mean.c_str(), -1, SQLITE_STATIC);

        //执行插入语句
        if(sqlite3_step(stmt) != SQLITE_DONE)
        {
            cerr<<"数据插入失败"<<sqlite3_errmsg(dict_db_)<<endl;
            sqlite3_finalize(stmt);
            file.close();
            return false;
        }

        //程序执行至此，表示该单词已经导入到数据库了
    }
}

```

```

        //是否stmt的资源
        sqlite3_finalize(stmt);

    }

    //程序执行至此，表示，单词表导入成功
    file.close();

    cout<<"单词库导入成功"<<endl;
    return true;
}

/*
    执行sql语句函数
    参数1: 数据库句柄
    参数2: 要执行的sql语句
    参数3: 错误信息
*/
bool DatabaseManager::executeSQL(sqlite3 *db, const string &sql, char ** errmsg)
{
    if(sqlite3_exec(db, sql.c_str(), NULL, NULL, errmsg)!=SQLITE_OK)
    {
        cerr<<"sql执行失败: "<<*errmsg<<endl;
        //释放错误信息空间
        sqlite3_free(errmsg);
        return false;
    }

    return true;        //表示正常执行了sql语句
}

/*
    注册功能：注册时，将用户输入的账号和密码存放到usr数据表中，如果该用户名已经存在，则给出报错
    信息
    参数1: 用户名
    参数2: 密码
*/
bool DatabaseManager::registerUser(const string &name, const string &password)
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句：?表示通配符，用于后面的绑定，0表示默认注册时，所有用户都是离线状态
    const char * sql = "insert into usr values(?,?,0)";

    //预编译sql语句
    sqlite3_stmt * stmt = NULL;
    if(sqlite3_prepare_v2(usr_db_, sql, -1, &stmt, NULL) !=SQLITE_OK)
    {
        cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
        return false;
    }

    //预编译成功，可以给已经预编译成功的内容绑定参数
    sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 2, password.c_str(), -1, SQLITE_STATIC);
}

```

```

//执行插入操作
int result = sqlite3_step(stmt);
sqlite3_finalize(stmt);

//对执行结果进行判断：该用户名已存在（失败） 用户名不存在（注册成功）
if(result == SQLITE_CONSTRAINT)
{
    cerr<<"该用户名已经存在: "<<name<<endl;
    return false;          //违反了主键唯一性
}

//程序执行至此，表示注册成功
return result == SQLITE_DONE;
}

/*
  用户登录功能
  参数1: 用户名
  参数2: 登录密码
  参数3: 表示是否已经在线，如果已经在线，则登录失败，防止重复登录
*/

bool DatabaseManager::loginUser(const string &name, const string &password, bool
&is_online)
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句
    const char *sql = "select stage from usr where name=? and passwd=?";
    //预编译sql语句
    sqlite3_stmt * stmt = NULL;
    if(sqlite3_prepare_v2(usr_db_, sql, -1, &stmt, NULL) !=SQLITE_OK)
    {
        cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
        return false;
    }

    //预编译成功，可以给已经预编译成功的内容绑定参数
    sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 2, password.c_str(), -1, SQLITE_STATIC);

    //执行查询操作
    int result = sqlite3_step(stmt);

    //对执行结果进行判断
    if(result==SQLITE_ROW)
    {
        //获取用户状态：0表示离线，1表示在线
        is_online = sqlite3_column_int(stmt, 0)==1;

        //如果当前用户不在线，更新状态为在线
        if(!is_online)
        {
            sqlite3_finalize(stmt);    //先释放空间

```

```

        //准备sql语句
        const char *update_sql = "update usr set stage=1 where name=?";
        if(sqlite3_prepare_v2(usr_db_, update_sql, -1, &stmt, NULL) !=
SQLITE_OK)
        {
            cerr<<"sql准备失败: "<<sqlite3_errmsg(usr_db_)<<endl;
            return false;
        }
        //绑定信息
        sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);
        result = sqlite3_step(stmt);
    }
}

//释放空间
sqlite3_finalize(stmt);
return result == SQLITE_DONE;
}

/*
    用户退出功能：将该用户对对应的状态调整成0即可
    参数：用户名
*/
bool DatabaseManager::logoutUser(const string &name)
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句
    const char *sql = "update usr set stage=0 where name=?";
    //预编译sql语句
    sqlite3_stmt * stmt = NULL;
    if(sqlite3_prepare_v2(usr_db_, sql, -1, &stmt, NULL) !=SQLITE_OK)
    {
        cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
        return false;
    }

    //绑定信息
    sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);
    //执行更新操作
    int result = sqlite3_step(stmt);
    sqlite3_finalize(stmt);

    return result == SQLITE_DONE;
}

/*
    查单词功能
    参数1：要查找的单词
    参数2：单词的含义
*/
bool DatabaseManager::queryword(const string &word, string &meaning)
{

```

```

//使用互斥锁锁定数据库的操作，防止出现多线程竞态
lock_guard<mutex> lock(dict_mutex_);

//准备sql语句
const char *sql = "select mean from dict where word=?";
//预编译sql语句
sqlite3_stmt * stmt = NULL;
if(sqlite3_prepare_v2(dict_db_, sql, -1, &stmt, NULL) !=SQLITE_OK)
{
    cerr<<"sql准备失败"<<sqlite3_errmsg(dict_db_)<<endl;
    return false;
}

//绑定信息
sqlite3_bind_text(stmt, 1, word.c_str(), -1, SQLITE_STATIC);

//执行查询
bool found = false;           //标识是否查询到
if(sqlite3_step(stmt) == SQLITE_ROW)
{
    //获取查询结果
    const unsigned char *result = sqlite3_column_text(stmt, 0);
    if(result)
    {
        meaning = reinterpret_cast<const char*>(result);    //转换数据类型

        found = true;           //标识找到了
    }
}

//释放空间
sqlite3_finalize(stmt);

return found;
}

/*
历史记录函数：将给定的信息插入到历史数据库中
参数1：用户名
参数2：单词
参数3：含义
参数4：查询时间
*/
bool DatabaseManager::recordHistory(const string &name, const string &word,
const string &meaning, const string &time)
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句
    const char *sql = "insert into history values(?,?,?,?)";
    //预编译sql语句
    sqlite3_stmt *stmt = NULL;
    if(sqlite3_prepare_v2(usr_db_, sql, -1, &stmt, NULL) != SQLITE_OK)
    {

```

```

        cerr<<"sql准备失败"<<sqlite3_errmsg(usr_db_)<<endl;
        return false;
    }

    //绑定信息
    sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 2, word.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 3, meaning.c_str(), -1, SQLITE_STATIC);
    sqlite3_bind_text(stmt, 4, time.c_str(), -1, SQLITE_STATIC);

    //执行插入操作
    int result = sqlite3_step(stmt);
    sqlite3_finalize(stmt);

    return result==SQLITE_DONE;
}

/*
    获取查单词的历史记录
    参数1: 用户名
    参数2: 查询的结果字符串
*/
bool DatabaseManager::getHistory(const string name, string &history)
{
    //使用互斥锁锁定数据库的操作，防止出现多线程竞态
    lock_guard<mutex> lock(usr_mutex_);

    //准备sql语句
    const char *sql = "select word,mean,time from history where name=? order by
time DESC;";
    //预编译sql语句
    sqlite3_stmt *stmt = NULL;
    if(sqlite3_prepare_v2(usr_db_, sql, -1, &stmt, NULL) != SQLITE_OK)
    {
        cerr<<"sql准备失败"<<sqlite3_errmsg(usr_db_)<<endl;
        return false;
    }

    //绑定信息
    sqlite3_bind_text(stmt, 1, name.c_str(), -1, SQLITE_STATIC);

    //执行查询语句
    history.clear();          //防止之前的数据干扰
    while(sqlite3_step(stmt) == SQLITE_ROW)
    {
        //获取该信息的每一列
        const unsigned char * word = sqlite3_column_text(stmt, 0);          //获取查
        询的单词
        const unsigned char * mean = sqlite3_column_text(stmt, 1);          //获取查
        询的含义
        const unsigned char * time = sqlite3_column_text(stmt, 2);          //获取查
        询的时间

        //将数据组成字符串
        if(word && mean && time)
        {

```

```

        history += string(reinterpret_cast<const char *>(word)) + "\t";
        history += string(reinterpret_cast<const char *>(mean)) + "\t";
        history += string(reinterpret_cast<const char *>(time)) + "\n";
    }
}

sqlite3_finalize(stmt);
return true;

}

```

## 2> database\_manager.hpp

```

#ifndef DATABASE_MANAGER_H
#define DATABASE_MANAGER_H

#include<myhead.h>
#include<sqlite3.h>           //数据库的头文件
#include<string>              //字符串头文件
#include<mutex>               //互斥锁头文件
#include<memory>              //智能指针所在的头文件

class DatabaseManager
{
public:
    //构造函数：需要外部传入用户库的路径以及单词库的路径
    DatabaseManager(const string &usr_db_path, const string &dict_db_path);
    ~DatabaseManager();

    //初始化数据表
    bool initializeDatabase();

    //用户相关的操作
    bool registerUser(const string &name, const string &password);           //用
户注册操作
    bool loginUser(const string &name, const string &password, bool
&is_online); //用户登录操作
    bool logoutUser(const string &name);                                     //用户退出操
作

    //单词表的操作
    bool queryWord(const string &word, string &meaning); //查询单词
    bool recordHistory(const string &name, const string &word, const string
&meaning, const string &time); //历史记录
    bool getHistory(const string name, string &history);

private:
    sqlite3 * usr_db_;           //用户数据库指针
    sqlite3 * dict_db_;          //单词数据库指针
    mutex usr_mutex_;            //用户库互斥锁
    mutex dict_mutex_;           //单词库互斥锁

    //定义私有函数
    bool initializeUserDB();      //初始化用户表

```

```

    bool initializeDictDB();           //初始化单词表
    bool dictToDatabase();            //导入词典数据

    //执行sql语句的函数
    bool executesQL(sqlite3 *db, const string &sql, char ** errmsg =NULL);

};

#endif

```

3> main.cpp

```

#include "database_manager.hpp"
#include "server.hpp"                //服务器的头文件

#include <myhead.h>

int main(int argc, const char *argv[])
{
    //判断外部是否传入的三个参数
    if(argc != 3)
    {
        cerr<<"用法: "<< argv[0]<<"<IP> <port>"<<endl;
        return -1;
    }

    try
    {
        //构造一个数据库管理对象，用于管理两个数据库
        auto db_manager = make_shared<DatabaseManager>("usr.db", "dict.db");
        //初始化数据库
        if(!db_manager->initailizeDatabase())
        {
            cerr<<"数据库初始化失败，请检查: "<<endl;
            cerr<<"1、dict.txt文件是否存在"<<endl;
            cerr<<"2、当前目录是否具有写权限"<<endl;
            return -1;
        }

        //构造一个服务器
        Server server(db_manager, argv[1], atoi(argv[2]));
        //启动服务器
        server.start();

    }
    catch(const std::exception& e)
    {
        std::cerr << e.what() << endl;
        return -1;
    }
}

```

```

        std::cout << "Hello, world!" << std::endl;
        return 0;
    }

```

4> message.hpp

```

#ifndef MESSAGE_HPP
#define MESSAGE_HPP

#include<myhead.h>

//操作类型的定义
#define R 1      //用户注册
#define L 2      //用户登录
#define Q 3      //用户退出
#define S 4      //单词查询
#define H 5      //历史记录

//定义客户端与服务器进行数据通信的协议
struct Msg
{
    int type;      // 操作类型
    char name[20]; //用户名
    char text[128]; //文本内容    (密码/单词)

    //定义相关字节序的转换函数
    //主机字节序转网络字节序
    void networkByteOrder()
    {
        type = htonl(type);
    }

    //网络字节序转主机字节序
    void hostByteOrder()
    {
        type = ntohl(type);
    }
};

#endif

```

5> server.cpp

```

#include"server.hpp"      //服务器头文件
#include"message.hpp"     //消息协议头文件
#include<myhead.h>
#include<time.h>           //有关时间的头文件
#include<thread>           //C++线程支持库

/*
    服务器的构造函数：完成套接字的创建、端口号快速重用、绑定ip和端口号

```

```

*/
Server::Server(shared_ptr<DatabaseManager> db_manager, const string ip, int
port):
    db_manager_(db_manager), ip_(ip), port_(port), running_(false)
//初始化列表
{
    //创建用于连接的套接字文件描述符
    sfd_ = socket(AF_INET, SOCK_STREAM, 0);
    if(sfd_ < 0)
    {
        throw runtime_error("socket error");    //抛出异常
    }

    //设置端口号快速重用
    int opt = 1;
    if(setsockopt(sfd_, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0)
    {
        close(sfd_);
        throw runtime_error("setsockopen error");
    }

    //绑定ip地址和端口号
    sockaddr_in sin;
    sin.sin_family = AF_INET;    //通信域
    sin.sin_port = htons(port);    //端口号
    sin.sin_addr.s_addr = inet_addr(ip.c_str());    //ip地址

    if(bind(sfd_, (sockaddr *)&sin, sizeof(sin))<0)
    {
        close(sfd_);
        throw runtime_error("bind error");
    }
}

/*
析构函数：停止服务器运行
*/
Server::~~Server()
{
    stop();
}

/*
启动服务器函数
*/
bool Server::start()
{
    //启动监听
    if(listen(sfd_, 5)<0)
    {
        cerr<<"listen error"<<endl;
        return false;
    }

    //设置运行状态
    running_ = true;    //表示可以接收客户端请求了
    cout<<"Server started on "<<ip_<<":"<<port_<<endl;
}

```

```

//等待客户端的连接请求：循环完成
while (running_)
{
    sockaddr_in client_addr;          //接收客户端地址信息结构体
    socklen_t addr_len = sizeof(client_addr);    //接收客户端地址信息结构体的大小

    //接收客户端连接请求操作
    int cfd = accept(sfd_, (struct sockaddr*)&client_addr, &addr_len);
    if(cfd < 0)
    {
        if(running_)
        {
            cerr<<"accept error"<<endl;
        }
        continue;
    }

    //创建一个分支线程，在分支线程中，进行跟客户端的通信
    thread([this, cfd, client_addr](){
        cout<<"Client connected : " << inet_ntoa(client_addr.sin_addr)<<":"<<
        <<ntohs(client_addr.sin_port)<<endl;

        //调用客户端处理函数
        handleClient(cfd, client_addr);
    }).detach() ;    //将线程设置成分离态，资源由系统回收

}

return true;
}

/*
    停止服务器函数
    //关闭套接字文件描述符
*/
bool Server::stop()
{
    if(running_)
    {
        running_ = false;    //非运行
        close(sfd_);    //关闭服务器套接字
        cout<<"Server stopped"<<endl;
        return true;
    }
    return false;
}

/*
    处理客户端请求，分为不同的请求模式，接收客户端消息，根据不同类型进行不同的处理
    参数1：客户端套接字文件描述符
    参数2：客户端地址信息结构体
*/
void Server::handleClient(int cfd, sockaddr_in client_addr)
{

```

```

Msg msg;           //用于存储收发的数据

while(true)
{
    //接收客户端消息
    ssize_t recv_len = recv(cfd, &msg, sizeof(msg), 0);
    if(recv_len <= 0)
    {
        if(recv_len == 0)           //当前客户端已经下线
        {
            cout<<"Client disconnect : "<< inet_ntoa(client_addr.sin_addr)
<<endl;
        }else
        {
            cerr<<"recv error"<<endl;
        }
        break;
    }

    //程序执行至此，表示正常读取到客户端的消息
    //转换字节序
    msg.hostByteOrder();

    //准备响应消息
    Msg response;           //用于回复消息的容器
    strncpy(response.name, msg.name, sizeof(response.name));           //表示要给谁
    传递消息
    response.type = msg.type;           //回复的消息类型

    //根据不同的消息请求处理不同的响应
    switch(msg.type)
    {
        case R:           //注册
        {
            //执行注册操作
            int success = db_manager->registerUser(msg.name, msg.text);
            strcpy(response.text, success? "***OK**" : "***EXISTS**");
            //如果成功注册回复OK，否则回复EXISTS
            break;
        }

        case L:           //登录
        {
            //执行登录操作
            bool is_online = false;           //返回是否已经在线
            bool success = db_manager->loginUser(msg.name, msg.text,
            is_online);           //执行登录函数
            if(success)
            {
                strcpy(response.text, is_online? "***EXISTS**":"***OK**");
                //根据是否已经在线，进行回复
            }else
            {
                strcpy(response.text, "***FAIL**");           //表示登录失败
            }
            break;
        }
    }
}

```

```

case Q:          //退出
{
    //执行退出操作
    db_manager_>logoutUser(msg.name);
    close(cfd);
    break;
}

case S:          //查询
{
    //执行查单词操作
    string meaning;          //报错单词含义
    if(db_manager_>queryword(msg.text, meaning))
    {
        //成功找到，将单词和含义一起回复给客户端
        snprintf(response.text, sizeof(response.text), "%s %s",
msg.text, meaning.c_str());
    }else
    {
        strcpy(response.text, "Not Found");          //表示没找到
    }

    //保存查单词记录
    time_t now = time(NULL);          //获取当前系统时间对应的秒数
    tm *local = localtime(&now);      //将以秒数为单位的时间转变成时间的结构
体

    char time_str[20] = "";          //保存当前系统时间的字符串
    strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S",
local);          //将结构体中的年月日时分秒转变成格式串

    //调用历史查询记录
    db_manager_>recordHistory(msg.name, msg.text, meaning,
time_str);

    break;
}

case H:          //历史记录
{
    //执行查看历史记录操作
    string history;          //保存得到的历史记录
    if(db_manager_>getHistory(msg.name, history))
    {
        //表示执行成功，history中就有了历史记录
        strncpy(response.text, history.c_str(),
sizeof(response.text));          //将历史记录发送给客户端
    }else
    {
        strcpy(response.text, "No history");          //当前用户没有进行查
单词操作
    }
    break;
}

default:strcpy(response.text, "Invalid Command");          //表示请求非法
}

//发送请求

```

```

        response.networkByteOrder();           //转变成网络字节序
        if(send(cfd, &response, sizeof(response), 0) < 0)
        {
            cerr<<"send error"<<endl;
            break;
        }

        //判断如果是查看历史记录，那么最后还要给出一个历史查询结束的情况
        if(msg.type == H)
        {
            strcpy(response.text, "**OVER**");
            send(cfd, &response, sizeof(response), 0);
        }
    }

    close(cfd);           //关闭客户端套接字
}

//获取当前系统时间的静态成员函数
string Server::getCurrentTime()
{
    //保存查单词记录
    time_t now = time(NULL);           //获取当前系统时间对应的秒数
    tm *local = localtime(&now);       //将以秒数为单位的时间转变成时间的结构体
    char time_str[20] = "";           //保存当前系统时间的字符串
    strftime(time_str, sizeof(time_str), "%Y-%m-%d %H:%M:%S", local); //将结构
    体中的年月日时分秒转变成格式串

    return string(time_str);
}

```

6> server.hpp

```

#ifndef SERVER_HPP
#define SERVER_HPP

#include "database_manager.hpp"           //数据库相关头文件
#include<memory>                         //智能指针头文件
#include<myhead.h>

//定义电子词典服务器类
class Server
{
public:
    //构造函数：需要使用数据库管理对象、ip地址、端口号进行构造一个服务器
    Server(shared_ptr<DatabaseManager> db_manager, const string ip, int port);
    ~Server();                           //析构函数

    //启动服务器
    bool start();
    //停止服务器
    bool stop();

private:
    int sfd_;                           //服务器套接字文件描述符
    int port_;                           //端口号

```

```

string ip_;           //ip地址
bool running_;       //运行标识
shared_ptr<DatabaseManager> db_manager_;    //数据库管理器

//处理客户端函数
void handleClient(int cfd, sockaddr_in client_addr);

//提供一个获取当前系统时间的函数
static string getCurrentTime();

};

#endif

```

7> CMakeLists.txt

```

cmake_minimum_required(VERSION 3.12)
project(dict_server CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(SQLite3 REQUIRED)
find_package(Threads REQUIRED)

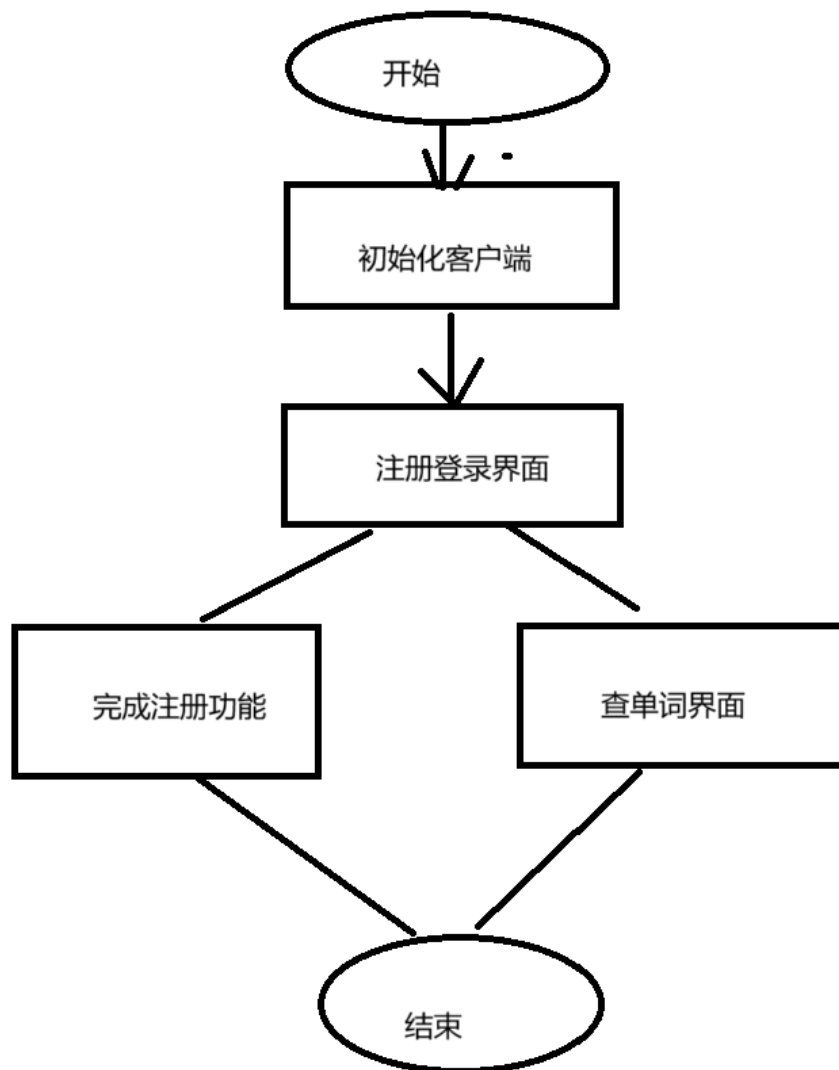
add_executable(server
    src/main.cpp
    src/server.cpp
    src/database_manager.cpp
)

target_include_directories(server PRIVATE ${CMAKE_SOURCE_DIR}/src)
target_link_libraries(server PRIVATE SQLite::SQLite3 Threads::Threads)

```

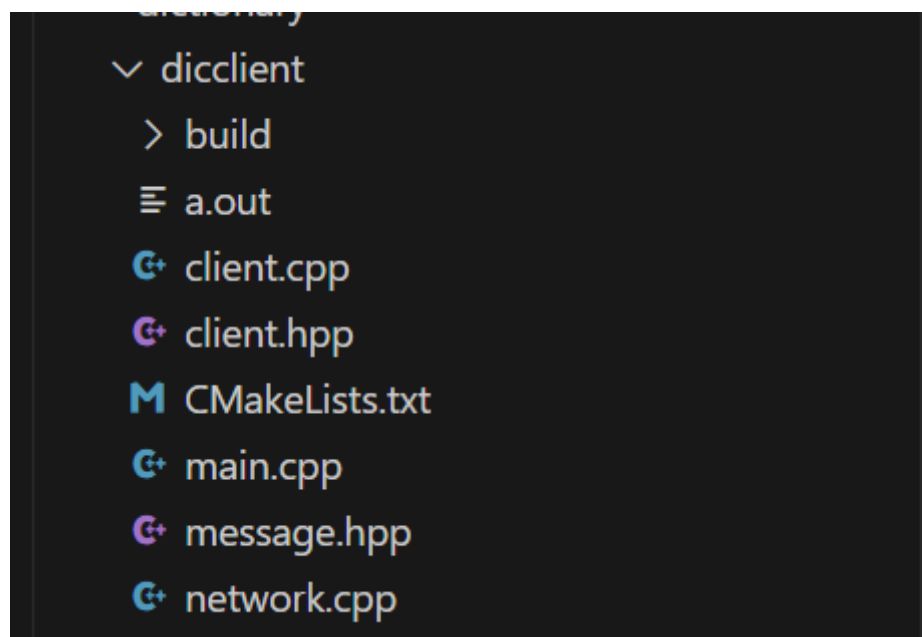
## 三、网络电子词典客户端实现

### 3.1 流程设计



## 3.2 代码实现

1> 项目逻辑



2> client.cpp

```
#include "client.hpp"
```

```

#include<myhead.h>

//主程序循环
void DictClient::run()
{
    while(true)
    {
        showMainMenu();           //展示主菜单
        int choice;               //选择功能
        cin >> choice;
        cin.ignore();             //清除缓冲区

        //对选则的内容进行多分枝
        switch (choice)
        {
            case R:                //注册功能
                doRegister();      //调用注册函数
                break;

            case L:                //登录功能
                if(doLogin())      //调用登录函数
                {
                    showUserMenu(); //调用用户展示函数
                }
                break;

            case Q:                //退出功能
                doQuit();          //调用退出函数
                break;

            default: cout<<"无效的选择"<<endl;
                    break;
        }

        cout<<"按回车继续....";
        cin.get();                //吸收回车
    }
}

//主菜单函数
void DictClient::showMainMenu()
{
    system("clear");             //用于清屏
    cout<<"*****"<<endl;
    cout<<"*****1、注册*****"<<endl;
    cout<<"*****2、登录*****"<<endl;
    cout<<"*****3、退出*****"<<endl;
    cout<<"*****"<<endl;
    cout<<"请选择: ";
}

//显示用户菜单
void DictClient::showUserMenu()
{
    while (is_logged_in_)
    {

```

```

system("clear");           //清屏
cout<<"当前用户: "<<username_<<endl;
cout<<"*****"<<endl;
cout<<"*****1、查单词*****"<<endl;
cout<<"*****2、历史记录*****"<<endl;
cout<<"*****3、返回上一级*****"<<endl;
cout<<"*****"<<endl;
cout<<"请选择: ";

int choice;
cin>>choice;
cin.ignore();

//对选择内容进行多分枝
switch (choice)
{
case 1:
    doQuery();           //调用查单词函数
    break;

case 2:
    doHistory();         //调用查单词函数
    break;

case 3:
    doQuit();            //调用查单词函数
    break;

default: cout<<"无效的选择"<<endl;
        break;
}

cout<<"按回车继续...."<<endl;
cin.get();
}

}

//用户注册函数
bool DictClient::doRegister()
{
    //定义相关数据信息
    Msg msg;
    msg.type = R;         //表示是注册类型

    //提示并输入用户名和密码
    cout<<"请输入用户名称(最长19个字符):";
    cin.getline(msg.name, sizeof(msg.name));
    cout<<"请输入密码(最长127个字符):";
    cin.getline(msg.text, sizeof(msg.text));

    //将上面的信息转换成网络字节序
    msg.networkByteOrder();
    //发送给服务器
    if(send(sockfd_, &msg, sizeof(msg), 0) < 0)
    {
        perror("注册请求失败");
    }
}

```

```

        return false;
    }

    //程序执行至此，表示注册成功
    if(recv(sockfd_, &msg, sizeof(msg), 0) <= 0)
    {
        perror("接收注册信息响应失败");
        return false;
    }
    //说明收到服务器的消息
    msg.hostByteOrder(); //将收到的服务器消息转换为本机字节序

    //处理服务器给出的响应
    if(strcmp(msg.text, "**OK**") == 0)
    {
        cout<<"注册成功"<<endl;
        return true;
    }else if(strcmp(msg.text, "**EXISTS**") == 0)
    {
        cout<<"注册失败，用户名已经存在"<<endl;
    }else
    {
        cout<<"注册失败，错误未知"<<endl;
    }

    return false;
}

//定义登录函数
bool DictClient::doLogin()
{
    //定义相关数据信息
    Msg msg;
    msg.type = L; //表示是登录类型

    cout<<"请输入用户名:";
    cin.getline(msg.name, sizeof(msg.name));
    cout<<"请输入密码: ";
    cin.getline(msg.text, sizeof(msg.text));

    //转换为网络字节序
    msg.networkByteOrder();
    //发送给服务器
    if(send(sockfd_, &msg, sizeof(msg) , 0) < 0)
    {
        perror("发送登录请求失败");
        return false;
    }

    //程序执行至此，表示向服务器发送数据成功，接收服务器反馈的结果
    if(recv(sockfd_, &msg, sizeof(msg), 0) <= 0)
    {
        perror("接收登录响应失败");
        return false;
    }
}

```

```

//将接收的反馈信息转换为本机字节序
msg.hostByteOrder();

//对接收的结果进行处理
if(strcmp(msg.text, "***OK**") == 0)
{
    cout<<"登录成功"<<endl;
    username_ = msg.name;
    is_logged_in_ = true;          //表示在线

    return true;
}else if(strcmp(msg.text, "***EXISTS**") == 0)
{
    cout<<"登录失败，用户已经在线"<<endl;
}else
{
    cout<<"登录失败，原因未知"<<endl;
}

return false;
}

//查单词功能
void DictClient::doQuery()
{
    Msg msg;          //信息传递体
    msg.type = S;      //表示查询功能
    strcpy(msg.name, username_.c_str()); //要查单词的用户名

    //循环进行查询单词
    while(true)
    {
        cout<<"请输入要查询的单词(输入#后结束查询):";
        cin.getline(msg.text, sizeof(msg.text));

        //判断是否结束查询
        if(strcmp(msg.text, "#") == 0)
        {
            cout<<"*****";
            break;
        }

        //将消息发送给服务器，进行检索
        msg.networkByteOrder();
        if(send(sockfd_, &msg, sizeof(msg), 0) < 0)
        {
            perror("发送查询消息失败");
            return ;
        }

        //函数执行至此，表示插叙成功，并收到服务器反馈的结果
        if(recv(sockfd_, &msg, sizeof(msg), 0) <= 0)
        {
            perror("接收查询结果失败");
        }
    }
}

```

```

        return;
    }
    msg.hostByteOrder();          //转化为主机字节序

    cout<<"释义: "<<msg.text<<endl;

}
}

//查看历史记录
void DictClient::doHistory()
{
    Msg msg;          //信息传递体
    msg.type = H;      //表示查询功能
    strcpy(msg.name, username_.c_str());    //要查单词的用户名

    //转换为网络字节序
    msg.networkByteOrder();
    if(send(sockfd_, &msg, sizeof(msg), 0) < 0)
    {
        perror("发送历史记录请求失败");
        return ;
    }

    cout<<"查询历史记录: "<<endl;
    while(true)
    {
        if(recv(sockfd_, &msg, sizeof(msg), 0) <=0)
        {
            perror("接收历史数据失败");
            return;
        }
        msg.hostByteOrder();          //转换为主机字节序

        //判断是否查询完毕
        if(strcmp(msg.text, "***OVER***") == 0)
        {
            break;
        }

        //将消息展示出来
        cout<<msg.text<<endl;
    }
}

//退出登录功能
void DictClient::doQuit()
{
    if(is_logged_in_)    //判断是否在线
    {
        Msg msg;          //信息传递体
        msg.type = Q;      //表示查询功能
        strcpy(msg.name, username_.c_str());    //要查单词的用户名

        //转换为网络字节序
        msg.networkByteOrder();
    }
}

```

```

        if(send(sockfd_, &msg, sizeof(msg), 0)<0)
        {
            perror("发送突出请求失败");
            return ;
        }else
        {
            is_logged_in_ = false;    //表示下线
            username_.clear();
            cout<<"用户已经退出"<<endl;
        }
    }
}
}

```

3> client.hpp

```

#ifndef CLIENT_H
#define CLIENT_H

#include"message.hpp"    //消息协议头文件

class DictClient
{
public:
    DictClient(const string &ip, int port);    //构造函数
    ~DictClient();    //析构函数
    void run();    //主程序运行函数

private:
    int sockfd_;    //客户端套接字文件描述符
    string username_;    //用户名
    bool is_logged_in_;    //判断是否已经登录

    //网络连接函数
    bool connectToServer(const string &ip, int port);

    //业务功能函数
    void showMainMenu();    //展示主菜单
    void showUserMenu();    //展示业务菜单
    bool doRegister();    //注册功能
    bool doLogin();    //登录功能
    void doQuery();    //查询功能
    void doHistory();    //获取历史记录功能
    void doQuit();    //退出功能
};

#endif

```

3> main.cpp

```

#include "client.hpp"

#include <myhead.h>

int main(int argc, const char *argv[])
{
    //判断传入的数据是否足够
    if(argc != 3)
    {
        cerr<<"用法: "<<argv[0]<<" <服务器ip> <端口号>"<<endl;
        return -1;
    }

    //实例化客户端对象
    try
    {
        DictClient client(argv[1], atoi(argv[2]));           //实例化客户端对象
        client.run();                                         //运行主要逻辑
    }
    catch(const std::exception& e)
    {
        std::cerr << e.what() << '\n';
    }

    std::cout << "Hello, world!" << std::endl;
    return 0;
}

```

#### 4> message.hpp

```

#ifndef MESSAGE_HPP
#define MESSAGE_HPP

#include<myhead.h>

//操作类型的定义
#define R 1           //用户注册
#define L 2           //用户登录
#define Q 3           //用户退出
#define S 4           //单词查询
#define H 5           //历史记录

//定义客户端与服务器进行数据通信的协议
struct Msg
{
    int type;          // 操作类型
    char name[20];      //用户名
    char text[128];     //文本内容    （密码/单词）

    //定义相关字节序的转换函数
    //主机字节序转网络字节序
    void networkByteOrder()
    {

```

```

        type = htonl(type);
    }

    //网络字节序转主机字节序
    void hostByteOrder()
    {
        type = ntohl(type);
    }
};

#endif

```

5> network.cpp

```

#include "client.hpp"
#include <stdexcept>           //异常处理类

//构造函数的定义
DictClient::DictClient(const string &ip, int port):sockfd_(-1),
is_logged_in_(false)
{
    //完成网络的连接功能
    if(!connectToServer(ip, port))
    {
        throw runtime_error("connect to server error");
    }
}

//析构函数
DictClient::~DictClient()
{
    //判断客户端是否存在
    if(sockfd_ >= 0)
    {
        doQuit();           //退出用户
        close(sockfd_);      //关闭套接字
    }
}

//网络连接函数的定义
bool DictClient::connectToServer(const string &ip, int port)
{
    //创建通信的套接字文件描述符
    sockfd_ = socket(AF_INET, SOCK_STREAM, 0);
    if(sockfd_ < 0)
    {
        perror("socket error");
        return false;
    }

    //设置端口号快速重用

```

```

int opt = 1;
if(setsockopt(sockfd_, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0)
{
    perror("setsockopt error");
    return false;
}

//连接服务器
sockaddr_in sin;          //服务器地址信息结构体
sin.sin_family = AF_INET;    //通信域
sin.sin_port = htons(port);  //端口号
sin.sin_addr.s_addr = inet_addr(ip.c_str());    //ip地址

//连接服务器
if(connect(sockfd_, (struct sockaddr*)&sin, sizeof(sin)) < 0)
{
    perror("connect error");
    return false;
}

return true;
}

```

6> CMakeLists.txt

```

cmake_minimum_required(VERSION 2.8)
project(dict_client)

# 设置C++标准
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 查找SQLite3（客户端可能不需要，但保留给未来扩展）
find_package(SQLite3)

# 查找线程库
find_package(Threads REQUIRED)

# 添加可执行文件（直接引用当前目录的源文件）
add_executable(${PROJECT_NAME}
    client.cpp
    network.cpp
    main.cpp
)

# 包含当前目录
target_include_directories(${PROJECT_NAME} PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}
)

# 链接库

```

```
target_link_libraries(${PROJECT_NAME} PRIVATE
    Threads::Threads
)

# 设置调试编译选项
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
    target_compile_options(${PROJECT_NAME} PRIVATE -Wall -wextra -g)
endif()
```